

A STORY-BASED PRACTICAL GUIDE

Git & GitHub From Zero to Confident Developer

Follow Arjun, Priya, Rahul, and Meena as they build real college projects — and learn version control the way working developers actually use it.

FOR FIRST-YEAR STUDENTS • NO PRIOR EXPERIENCE REQUIRED

featuring Arjun · Priya · Rahul · Meena · and Kabir, their senior mentor

Table of Contents

Every entry below is a clickable link in your PDF reader.

PART I — THE BIG PICTURE

Chapter 1 — Before Git: The Problem	7
<i>Why every team eventually invents version control, whether they know it or not</i>	
Chapter 2 — Installing and Configuring Git	11
<i>Getting Git onto your machine, and telling it who you are</i>	
Chapter 3 — Understanding the Git Mental Model	15
<i>The one diagram that makes every future command make sense</i>	

PART II — MASTERING LOCAL GIT

Chapter 4 — Creating Your First Repository	19
<i>Turning an ordinary folder into a project Git can protect</i>	
Chapter 5 — Understanding git status	23
<i>Learning to read Git's most honest, most helpful command</i>	
Chapter 6 — git add	26
<i>Choosing exactly what belongs in your next commit</i>	
Chapter 7 — git commit	29
<i>Saving a snapshot your future self will thank you for</i>	
Chapter 8 — Viewing History	33
<i>Reading the story your commits have already written</i>	
Chapter 9 — Undoing Changes	37
<i>Five different kinds of 'undo', and how to pick the right one</i>	
Chapter 10 — .gitignore	42
<i>Teaching Git what to politely ignore</i>	

PART III — GOING REMOTE WITH GITHUB

Chapter 11 — Connecting Git to GitHub	46
<i>Giving your local repository a home on the internet</i>	
Chapter 12 — git push	50
<i>Sending your local commits out into the world</i>	
Chapter 13 — git clone	53
<i>Downloading a complete project, history included</i>	
Chapter 14 — git pull and git fetch	55
<i>Bringing a teammate's work onto your own machine</i>	

PART IV — BRANCHING AND TEAMWORK

Chapter 15 — Branches	60
<i>Working in parallel without stepping on each other's code</i>	
Chapter 16 — Merging	64
<i>Bringing two lines of history back together</i>	
Chapter 17 — Merge Conflicts	67
<i>What to do when Git genuinely can't decide for you</i>	
Chapter 18 — GitHub Pull Requests	70
<i>Asking for a second pair of eyes before code joins main</i>	
Chapter 19 — Forks	74
<i>Contributing to a project you don't have permission to edit directly</i>	

PART V — GITHUB PROJECT FEATURES

Chapter 20 — GitHub Issues	78
<i>Tracking bugs, ideas, and tasks without a single sticky note</i>	
Chapter 21 — GitHub README	80
<i>Writing the first (and sometimes only) thing anyone reads</i>	
Chapter 22 — Git Tags and Releases	82
<i>Marking the exact moment a version became official</i>	

PART VI — ADVANCED LOCAL GIT

Chapter 23 — Stashing	86
<i>Setting messy, half-finished work aside without committing it</i>	
Chapter 24 — Git Rebase	89
<i>Rewriting history to look like it happened in a straight line</i>	
Chapter 25 — Remote Management	92
<i>Working with more than one remote repository at once</i>	

PART VII — REAL-WORLD WORKFLOWS

Chapter 26 — GitHub Collaboration Workflow	96
<i>Putting every piece together on one real team project</i>	
Chapter 27 — GitHub Open Source Workflow	100
<i>Contributing to a project the world can see</i>	
Chapter 28 — GitHub Authentication	103
<i>Proving it's really you, safely</i>	
Chapter 29 — GitHub Actions: A Beginner Introduction	106
<i>Letting GitHub test your code automatically, every single time</i>	

PART VIII — TROUBLESHOOTING & MASTERY

Chapter 30 — Common Git Errors	109
<i>A field guide to the red text every developer eventually sees</i>	
Chapter 31 — Git Best Practices	112
<i>The habits that separate a confident developer from a dangerous one</i>	

PART IX — REFERENCE

Chapter 32 — Git Command Cheat Sheet	115
<i>Every command in this book, one line each</i>	
Chapter 33 — Command-by-Command Reference	118
<i>One detailed table, every command, side by side</i>	

PART X — PRACTICE & ASSESSMENT

Chapter 34 — Hands-On Labs	121
<i>Ten progressively harder exercises, from your first commit to a real contribution</i>	
Chapter 35 — Mini Project: College Club Management System	128
<i>One assignment that exercises everything this book has taught</i>	
Chapter 36 — Final Interview / Viva Questions	130
<i>Forty questions that separate 'used Git once' from 'understands Git'</i>	

How To Use This Book

Meet the cast, and learn the rhythm of every chapter

This is not a manual you are meant to memorise. It is a story you are meant to *live through* — one project, one mistake, and one "aha" moment at a time. Every Git and GitHub concept in this book is introduced through the experience of a small group of first-year college students building real projects together. You will make the same mistakes they make, ask the same questions they ask, and by the end, you will reach for Git commands the same way they do: without thinking twice.

The Cast

Character	Who they are
Arjun	A first-year Computer Science student. Enthusiastic, a little disorganised, and about to learn version control the hard way — by losing work he can't get back.
Priya	Arjun's teammate. Methodical and curious, she's the one who reads the error message before panicking.
Rahul	The team's backend developer. Confident, fast, and occasionally a little too comfortable with risky commands (you'll meet his mistakes in later chapters).
Meena	Joins the team later as they take on a bigger project. Great at reviewing other people's code before it ships.
Kabir	A final-year student who runs the college's coding club. Whenever the team is stuck, Kabir is the one who calmly explains what's really going on — and why.

The Pattern Behind Every Concept

Git has a reputation for being confusing because most tutorials teach commands before they teach the *problem those commands solve*. This book refuses to do that. Every major concept in this book follows the same eleven-step rhythm:

Step	What it gives you
1. Story	A scene from Arjun and the team's project — something you can picture.
2. Problem	The exact frustration that step in the story creates.
3. Concept	The plain-English idea that solves that frustration.
4. Technical Explanation	What Git is actually doing internally, without hand-waving.
5. Command / Syntax	The exact command that expresses the concept.
6. Real Terminal Example	A command exactly as you would type it.
7. Explanation of Each Part	What every word, flag, and argument in the command means.
8. Expected Output	What you should see on your own screen, so you know it worked.
9. When To Use It	The real-world situations where this command is the right tool.
10. Common Mistakes	The specific ways beginners (and experienced developers) get this wrong.
11. Mini Practice Task	Something small to actually type, right now, on your own machine.

TIP

You will get the most out of this book with a terminal open beside you. Reading about Git is a bit like reading about swimming — it only really clicks once you're in the water. Type every command yourself, even the ones that seem obvious.

NOTE

Commands shown for Windows use Git Bash (installed automatically with Git for Windows) so that the same commands work identically on Windows, macOS, and Linux throughout this book.

PART I

THE BIG PICTURE

Before Git: The Problem

Why every team eventually invents version control, whether they know it or not

Long before Arjun ever types `git init`, he already needs Git — he just doesn't know it yet.

A Normal Tuesday Night

THE STORY CONTINUES

It is 11:40 p.m. Arjun has been working on his first big college assignment, a simple **Student Attendance System**, for six hours straight. He has just finished a feature that calculates attendance percentage, and it finally works. Relieved, he does what every student has done at least once: he right-clicks the project folder, chooses Copy, pastes it, and renames the copy `project_final.zip`.

Feeling safe, he keeps experimenting. Twenty minutes later he breaks something while trying to "improve" the percentage calculation, and he is not sure which lines he changed. He makes another copy: `project_final_v2.zip`. By 1 a.m., his folder looks like this:

```
Arjun's Desktop
project_final.zip
project_final_v2.zip
project_final_REAL.zip
project_final_REAL_FINAL.zip
project_final_REAL_FINAL_USE_THIS_ONE.zip
```

THE PROBLEM

Arjun now has five folders and no idea which one actually contains his working percentage calculation. He cannot answer simple questions that any developer needs to answer every single day:

- Which version is the one that actually works?
- What exactly changed between `v2` and `REAL` ?
- If he deletes the older zips to save space, can he ever get that code back?
- If Priya and Rahul are also editing copies of this project, how do their changes ever come back together into one final version?

Kabir Drops By

THE STORY CONTINUES

Kabir, a final-year student who runs the college coding club, sees Arjun's screen in the lab and laughs — not unkindly. "I had a folder exactly like that in my first year," he says. "Every developer eventually invents the same bad solution to the same problem, until someone shows them the real one. What you're trying to do already has a name: version control. And the tool almost everyone uses for it is called Git."

What Is Version Control?

THE CONCEPT

Version control is a system that records changes to a set of files over time, so that you can recall specific versions later. Instead of manually copying entire folders every time you want a safety net, a version control system quietly keeps a complete, searchable history of every change — who made it, when, and exactly what was different — without ever needing a second copy of the whole project.

This idea is not new or exotic. It exists because the moment more than one person (or even one person working over time) touches the same files, three problems always appear:

Problem	What happens without version control
Losing history	Once you overwrite a file, yesterday's working version is gone unless you happened to save a copy of it.
No accountability	In a team, nobody can tell who changed what, or why, once changes are merged by hand.
Merging chaos	Combining two people's edits to the same file usually means manually reading both versions and retyping the differences.

Version Control Systems: Centralised vs. Distributed

There have been two broad generations of version control tools:

Type	How it works	Examples
Centralised (CVCS)	One central server holds the entire project history. Everyone connects to it to save or retrieve changes. If the server goes down, nobody can save history — and if its disk fails without backups, the history can be lost forever.	CVS, Subversion (SVN)
Distributed (DVCS)	Every single developer's computer holds a <i>complete</i> copy of the entire project history, not just the current files. There is no single point of failure, and most operations (viewing history, committing, branching) work instantly, without needing the internet at all.	Git, Mercurial

THE CONCEPT: DISTRIBUTED VERSION CONTROL

Git is a Distributed Version Control System (DVCS). When Arjun runs `git clone` on a project, he is not just downloading the current files — he is downloading the *entire history* of that project, every commit anyone has ever made. His laptop becomes a full, independent backup of the project's history, even before he writes a single line of code.

So, What Exactly Is Git?

THE CONCEPT: GIT

Git is a free, open-source piece of software that you install on your own computer. It runs entirely on your machine and does not require the internet. Its only job is to track changes to the files in a project folder over time, so that nothing is ever truly lost and every change can be inspected, compared, and (if necessary) undone.

So, What Exactly Is GitHub?

THE CONCEPT: GITHUB

GitHub is a website — an online platform — that hosts Git projects "in the cloud" and adds collaboration tools around them: a place to store a backup of your project online, request that a teammate review your code before it is accepted, track bugs and tasks, and showcase your work to the world. GitHub does not replace Git; it is built on top of Git.

NOTE

GitHub is the most popular platform of its kind, but it is not the only one. **GitLab** and **Bitbucket** are well-known alternatives that also host Git repositories with similar collaboration features. Everything you learn about Git itself in this book transfers directly to any of them — only the website around it changes.

Git vs. GitHub: The Google Docs Analogy

THE STORY CONTINUES

Kabir picks up a marker. "Think about Microsoft Word for a second," he says. "Word itself — the application on your laptop — lets you track changes and save different versions of a document, even with your Wi-Fi off. That's Git. Now, Google Docs is a website where those documents can live online, so that other people can open them, comment on them, and edit them with you. That's GitHub. One is the tool. The other is the website built around sharing what the tool produces."

	Git	GitHub
What it is	A tool / software	A website / online platform
Where it lives	Installed on your own computer	Runs on servers on the internet
Needs internet?	No — works fully offline	Yes — to push, pull, or browse projects online
Main job	Tracks changes to files (history)	Hosts that history online and adds collaboration: Pull Requests, Issues, code review
Made by	Linus Torvalds (2005), open source	GitHub, Inc. (now owned by Microsoft)
Could exist without the other?	Yes — Git worked for years before GitHub existed	No — GitHub is built entirely on top of Git repositories

New Vocabulary You Will Use Every Day

Before moving forward, here are five words Kabir uses constantly. You do not need to fully understand them yet — that is what the next two chapters are for — but you should recognise them on sight.

Term	In one sentence
Repository ("repo")	A project folder that Git is keeping track of — it contains your files <i>plus</i> their entire history of changes.
Commit	A saved snapshot of your project at one point in time, along with a message describing what changed.
Branch	An independent line of work, so you (or a teammate) can make changes without disturbing the main, stable version of the project.
Remote	A copy of the repository stored somewhere else — almost always on GitHub — that your local copy can send changes to or receive changes from.
Local repository	The copy of the repository living on your own computer, as opposed to the remote copy on GitHub.

COMMON MISTAKES

- Thinking Git and GitHub are the same thing — they are two different products made by different organisations.
- Believing you need an internet connection to use Git — you only need one to reach GitHub.
- Assuming version control is only useful for "big" projects with many developers — Arjun's single-person assignment already needed it on night one.
- Thinking you have to fully understand Git before starting — you learn Git by using it, the same way you learn to type by typing.

MINI PRACTICE TASK

Before you touch a keyboard for Git itself, do this reflection exercise:

1. Think of one real situation (an assignment, a personal project, even a resume) where you kept multiple renamed copies of the same file "just in case."
2. Write down, in one sentence, what problem you were actually trying to solve by doing that.
3. Keep that sentence handy — by the end of Chapter 4, you'll see exactly how Git solves it properly.

Installing and Configuring Git

Getting Git onto your machine, and telling it who you are

Arjun opens his laptop the next morning, ready to stop making zip files forever — but first, Git has to actually be installed.

Installing Git

Git is free and works identically in spirit on every major operating system, though the installation step looks slightly different on each one.

Windows

WINDOWS

1. Download the installer from `git-scm.com`.
2. Run the installer. The default options are safe for beginners — keep clicking "Next."
3. This installs **Git Bash**, a terminal that behaves like a Linux/macOS terminal, even on Windows. All commands in this book will work in Git Bash exactly as written.

macOS

MACOS

The simplest route is via the Terminal app, using Homebrew:

SYNTAX

```
brew install git
```

Alternatively, running `git --version` for the first time on a fresh Mac will prompt macOS to offer installing the Xcode Command Line Tools, which include Git.

Linux

LINUX

Use your distribution's package manager. On Debian/Ubuntu-based systems:

SYNTAX

```
sudo apt update
sudo apt install git
```

On Fedora-based systems, use `sudo dnf install git` instead.

```
git --version
```

THE STORY CONTINUES

Arjun installs Git and immediately wants proof it worked, the same way you'd check a new phone actually turns on.

THE CONCEPT

This command asks Git to report its own version number. It is the universal "is this installed correctly?" sanity check for any command-line tool, not just Git.

SYNTAX

```
git --version
```

```
terminal
$ git --version
git version 2.46.0
```

WHEN TO USE THIS

Run this right after installing Git, and again any time a tutorial or teammate mentions a feature that "requires a newer Git version."

COMMON MISTAKES

- Typing `git -v` or `git version` and being confused when it doesn't work the same way — the correct flag is `-version`.

Telling Git Who You Are

THE STORY CONTINUES

Arjun runs his first real Git command inside a project, and Git immediately stops him with an error demanding to know his identity. Kabir explains: "Every single commit you ever make is permanently stamped with a name and an email address. Git needs to know that information before it lets you commit anything, so it asks you to configure it once, globally, for your whole computer."

```
git config --global user.name "Your Name"
```

THE PROBLEM

Without an identity attached to a commit, nobody — not you, not a teammate, not a hiring manager looking at your GitHub profile — could ever tell who wrote which change.

THE CONCEPT

This command permanently stores your display name inside Git's configuration file on your computer, so it can be attached to every commit you make from now on, in every project.

WHAT GIT DOES INTERNALLY

The `--global` flag tells Git to write this setting into a configuration file stored in your home directory (`~/.gitconfig`), rather than inside one specific project. That means you only ever need to set it once per computer.

SYNTAX

```
git config --global user.name "Your Name"
```



```
terminal
```

```
$ git config --global user.name "Arjun Menon"
```

Part of the command	What it means
<code>git config</code>	Reads or changes Git's configuration settings.
<code>--global</code>	Apply this setting to every repository on this computer, not just the current one.
<code>user.name</code>	The specific setting being changed: your display name.
<code>"Arjun Menon"</code>	The value being stored, in quotes because it contains a space.

WHEN TO USE THIS

Run this exactly once per computer, immediately after installing Git — typically right alongside setting your email in the next section.

COMMON MISTAKES

- Forgetting the quotes around a name that contains a space, which causes Git to treat the second word as a separate, invalid argument.
- Setting this per-project by accident and forgetting `--global`, then wondering why a brand new project asks for identity again.

```
git config --global user.email "you@example.com"
```

Sets the email address attached to every commit, the same way `user.name` sets your display name. GitHub later uses this email to match your commits to your GitHub profile picture.

SYNTAX

```
git config --global user.email "you@example.com"
```

```
terminal
```

```
$ git config --global user.email "arjun.menon@example.com"
```

When to use it: Right after setting your name — the two are almost always configured back to back.

Watch out: Using a different email here than the one registered on your GitHub account. Git will still work, but GitHub may not link your commits to your profile.

```
git config --list
```

Displays every configuration setting Git currently knows about — global and local combined — so you can confirm your name and email were actually saved correctly.

SYNTAX

```
git config --list
```

terminal

```
$ git config --list
user.name=Arjun Menon
user.email=arjun.menon@example.com
init.defaultbranch=main
```

When to use it: Any time you want to double-check your setup, especially on a lab or shared computer.

`git config --global init.defaultBranch main`

Sets the name Git will use for the very first branch of any new project you create. Historically Git defaulted to a branch named `master`; the modern convention (and GitHub's default) is `main`, so setting this avoids a mismatch later.

SYNTAX

```
git config --global init.defaultBranch main
```

terminal

```
$ git config --global init.defaultBranch main
```

When to use it: Once, right after installing Git, so every future `git init` starts you on `main` automatically.

Watch out: Skipping this and later being confused why an old tutorial's screenshots say `master` but your project says `main` (or vice versa) — both are just names; neither is more “correct” technically, but `main` is the current convention.

COMMON MISTAKES

- Running project-specific commands before configuring `user.name` / `user.email`, then getting stuck on an identity error at the exact moment you're excited to make your first commit.
- Assuming `git config` changes need to be repeated for every new project — with `--global`, you only ever do this once per computer.

MINI PRACTICE TASK

Set up Git on your own machine right now:

1. Install Git for your operating system using the steps above.
2. Run `git --version` and confirm you see a version number.
3. Configure your name and email using `git config --global`.
4. Run `git config --list` and confirm both values appear correctly.

Understanding the Git Mental Model

The one diagram that makes every future command make sense

Before typing another command, Arjun needs a map. Every Git command he will ever run simply moves something between four places.

THE STORY CONTINUES

Kabir stops Arjun before he types anything else. "Everyone rushes to memorise commands," he says, "and then gets confused six months later because they never understood what the commands were actually doing. Give me two minutes for one diagram, and I promise every command in this book will make sense before you even learn it."

The Four Places Your Code Can Live

THE CONCEPT

At any moment, a piece of code you write exists in one of four places. Git's entire job is to help you move that code deliberately between them.

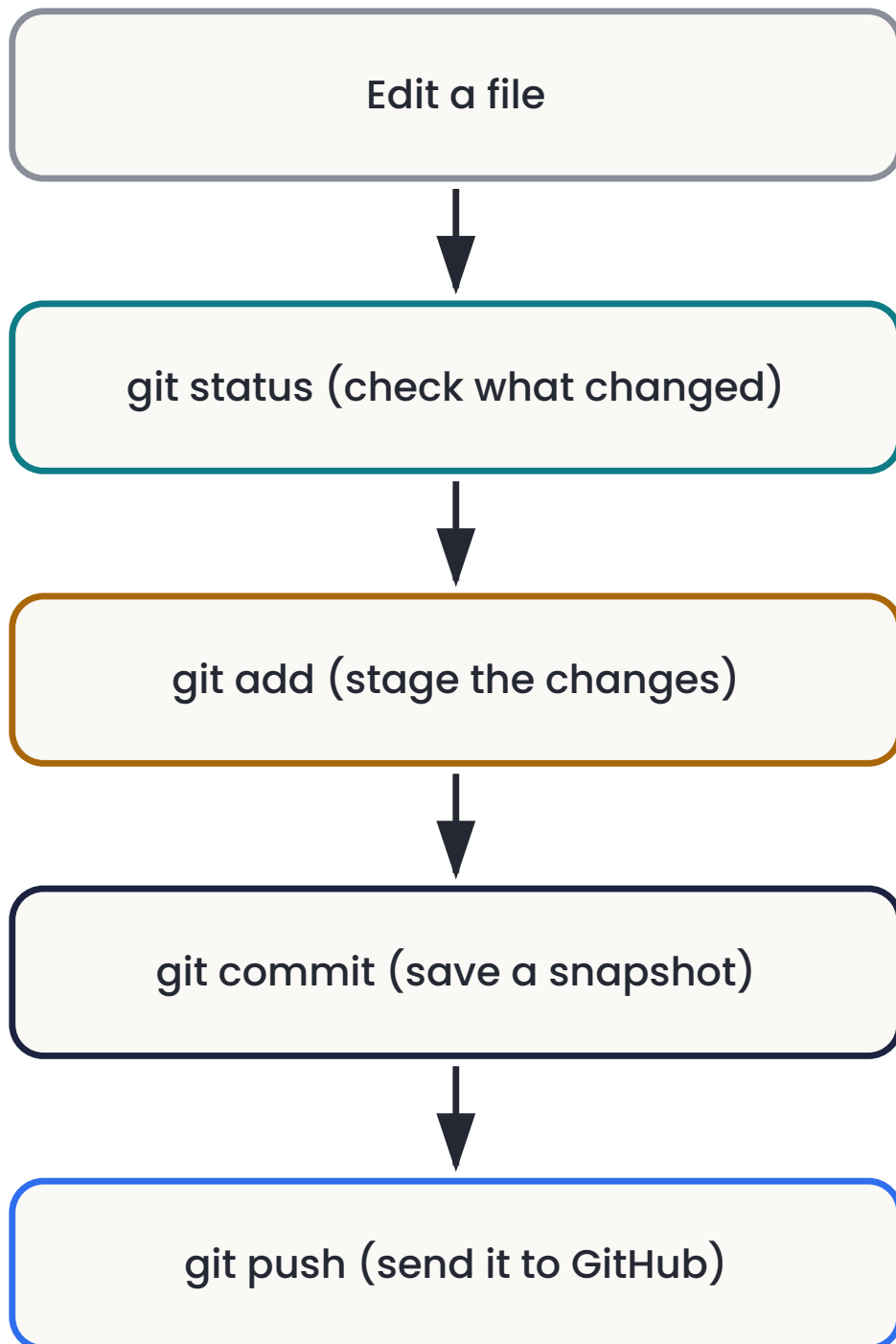


The four stages every change to your code passes through, and the command that moves it forward at each step.

Stage	What it actually is
Working Directory	The actual files on your computer, exactly as you see them in your code editor right now. This is where you edit, save, break, and fix your code.
Staging Area (the "index")	A waiting room. It holds a list of exactly which changes you have decided are ready to be included in your <i>next</i> commit — nothing more, nothing less.
Local Repository	The permanent, saved history of commits stored inside a hidden <code>.git</code> folder on your own computer. Nothing here is lost even if you have no internet connection.
Remote Repository	A copy of that same history stored somewhere else, almost always on GitHub, so that it survives even if your laptop is lost, and so teammates can access it too.

The Everyday Lifecycle

Almost every working session with Git follows the same rhythm:



The everyday Git rhythm: edit, check, stage, commit, push.

Why "git add" Does NOT Mean "Upload to GitHub"

THE PROBLEM

This is, by far, the most common beginner misunderstanding in all of Git. Students hear "staging" and assume it means their code has now left their laptop. It has not — it has not even left the `.git` folder.

THE CONCEPT

git add

Marks changes as “ready.”
Everything stays 100% local.
Nothing is saved to history yet,
and nothing leaves your
computer.

git commit

Permanently saves a snapshot of
the staged changes into your
local history. Still 100% local —
still nothing has left your
computer.

git push

The *only* one of the three that
touches the internet. It uploads
your local commits to the
remote repository on GitHub.

TIP

A useful rule of thumb for the rest of this book: **if a command doesn't say “push”, “pull”, “fetch”, or “clone”, it almost certainly never touches the internet.** Everything else — status, add, commit, log, branch, merge, stash, reset — happens entirely on your own machine.

HEAD and Commit History

THE CONCEPT

HEAD is Git's name for “whatever commit you currently have checked out” — in other words, a pointer to your current position in the project's history. Almost all of the time, HEAD simply points to the latest commit on whichever branch you are currently on. You will see the word HEAD constantly from Chapter 8 onward, especially when undoing changes.

COMMON MISTAKES

- Believing `git add` sends code to GitHub — it only stages changes locally.
- Believing `git commit` sends code to GitHub — it only saves a snapshot locally.
- Panicking when offline, thinking Git won't work at all — only `push` / `pull` / `fetch` / `clone` need the internet.

MINI PRACTICE TASK

Test your understanding before moving to Chapter 4:

1. Without opening a terminal, write down from memory the four stages a change passes through.
2. Write down which single command is the only one in the everyday lifecycle that requires internet access.
3. Explain in one sentence, in your own words, why staging and committing are two separate steps instead of one.

PART II

MASTERING LOCAL GIT

Creating Your First Repository

Turning an ordinary folder into a project Git can protect

Arjun opens a brand new, empty folder for the **Student Attendance System**. It's time to turn it into his first real Git repository.

Setting Up the Project Folder

Before Git enters the picture at all, Arjun needs a folder for the project to live in. These two commands are plain operating-system commands (not Git commands), but you will type them at the start of nearly every project in this book.

```
terminal
$ mkdir student-attendance-system
$ cd student-attendance-system
```

Part of the command	What it means
<code>mkdir</code>	"Make directory" — creates a new, empty folder.
<code>cd</code>	"Change directory" — moves your terminal session inside that folder.

TIP

Everything you type from this point on in this chapter happens **inside** this folder. If a command ever gives you a strange "not found" or "not a git repository" error, the very first thing to check is whether you're still in the right folder — running `pwd` (macOS/Linux) or `cd` alone (Windows) prints your current location.

git init

THE STORY CONTINUES

Arjun is sitting inside an empty `student-attendance-system` folder. Right now, as far as Git is concerned, this is just an ordinary folder — Git isn't watching it at all.

THE PROBLEM

If Arjun tries any Git command right now — for example `git status` — Git will refuse, because there is no repository here yet for it to report on.

THE CONCEPT

`git init` ("initialize") turns the current folder into a Git repository. It does this by creating a hidden subfolder named `.git` that will hold the project's entire future history. Nothing about your actual files changes — Git simply starts watching this folder.

WHAT GIT DOES INTERNALLY

Internally, `git init` creates the `.git` directory and populates it with the skeleton Git needs: an object database (initially empty), a `refs` folder to eventually hold branch pointers, and a `HEAD` file that will point to your current branch. No commit exists yet — you have an empty, ready-to-use repository.

SYNTAX

git init

terminal

```
$ git init
Initialized empty Git repository in /home/arjun/student-attendance-system/.git/
```

Part of the command	What it means
git	Tells the terminal you want to run a Git command.
init	The specific action: initialize a new, empty repository in the current folder.

WHEN TO USE THIS

Run `git init` exactly once, at the very beginning of a brand-new project that does not already exist on GitHub. (If a project *already* exists on GitHub and you want a local copy, you will use `git clone` instead — see Chapter 13.)

COMMON MISTAKES

- Running `git init` inside a folder that is already a Git repository (or nested inside one), accidentally creating a confusing repository-within-a-repository.
- Running it in the wrong folder because you forgot to `cd` into the project folder first — always confirm your location before initializing.
- Thinking `git init` connects you to GitHub in any way — it does not. It is a purely local, offline command.

Peeking Inside .git

THE STORY CONTINUES

Curious, Arjun runs `ls -a` and notices a folder he's never seen before: `.git`. Kabir warns him immediately: "That folder is the entire brain of your repository. Never edit anything inside it by hand — let Git commands manage it for you."

terminal

```
$ ls -a
.  ..  .git
$ ls .git
HEAD config description hooks info objects refs
```

WHAT GIT DOES INTERNALLY

A few of the most important pieces inside `.git`:

- `HEAD` — a small file that records which branch (and eventually, which commit) you currently have checked out.
- `objects/` — Git's database. Every version of every file you ever commit is stored here, compressed, forever.
- `refs/` — pointers to your branches and tags.
- `config` — settings specific to this one repository (as opposed to the global settings you set in Chapter 2).

⚠ DANGER ZONE

Never manually delete, rename, or edit files inside `.git` unless you know *exactly* what you are doing and why. Deleting the `.git` folder itself deletes your *entire project history* — permanently. Your current files on disk would survive, but every past commit would be gone forever.

Writing a README

THE CONCEPT

A `README.md` file is simply a plain text file, written in a lightweight formatting language called **Markdown**, that explains what a project is and how to use it. It is usually the very first file created in any new repository, and GitHub automatically displays its contents on the project's main page.

```
terminal
$ echo "# Student Attendance System" > README.md
$ cat README.md
# Student Attendance System
```

Your First Save: A Guided Walkthrough

Arjun is now ready to save his first snapshot. The next four commands — `git status`, `git add`, `git commit`, and `git log` — each get an entire chapter of their own very soon (Chapters 5 through 8). For now, walk through them once, end to end, so you can see the whole picture before we slow down and examine each piece under a microscope.

```
terminal
$ git status
On branch main

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
    README.md

nothing added to commit but untracked files present (use "git add" to track)

$ git add README.md
$ git status
On branch main

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
    new file:   README.md

$ git commit -m "Initial commit"
[main (root-commit) 3f2a91c] Initial commit
 1 file changed, 1 insertion(+)
 create mode 100644 README.md

$ git log --oneline
3f2a91c (HEAD -> main) Initial commit
```

NOTE

Notice that `git status` is talking to Arjun in plain English, even suggesting the next command to run. Reading `git status` output carefully is, by far, the single most useful debugging habit you can build as a beginner.

COMMON MISTAKES

- Running `git commit` before `git add`, then being confused that Git says "nothing to commit."
- Forgetting the `-m` flag on `git commit`, which drops you into a text editor (often Vim) that beginners don't know how to exit — press `Esc`, then type `:wq` and press Enter if this happens to you.
- Expecting `git init` in a folder that already has a GitHub repository to "connect" to it automatically — initializing and connecting to a remote are two separate steps (Chapter 11).

MINI PRACTICE TASK

Create your own first repository from scratch:

1. Create a new folder for a small project of your choice and move into it.
2. Run `git init` and confirm you see the "Initialized empty Git repository" message.
3. Create a `README.md` file with one line describing the project.
4. Run `git status`, then `git add`, then `git commit -m "Initial commit"`.
5. Run `git log --oneline` and confirm your commit appears.

Understanding git status

Learning to read Git's most honest, most helpful command

If Arjun learns to truly trust one command above all others, Kabir says it should be this one.

THE STORY CONTINUES

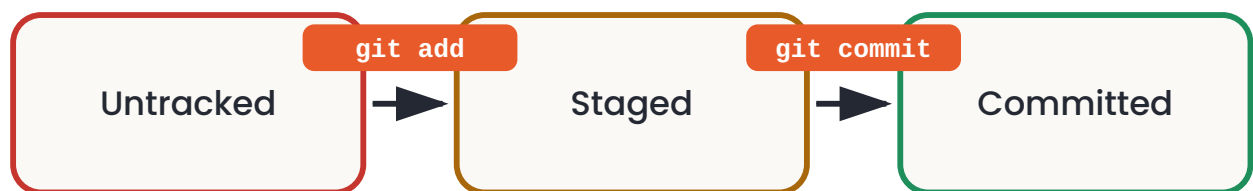
A week into the project, Arjun is juggling several changes at once: he has edited `attendance.py`, created a new file `utils.py`, and left `README.md` untouched. He has completely lost track of what state each file is in. "Stop guessing," Kabir says. "Just ask Git. It always knows exactly what state every file is in — you just have to ask it with `git status`."

The Four States a File Can Be In

THE CONCEPT

From Git's point of view, every file in your project folder is in exactly one of four states at any given moment:

State	Meaning
Untracked	Git has never seen this file before. It exists on disk, but Git is not watching it at all yet.
Modified	Git is already tracking this file, and you have changed it since the last commit — but you haven't staged those changes yet.
Staged	You have marked this file's current changes as ready to be included in the next commit, using <code>git add</code> .
Committed	The file's current content is safely saved in your project's history. There is nothing new to save.



The straight-line journey of a brand-new file. A tracked file that you edit again after committing becomes "Modified" and re-enters this cycle from the staged step.

git status

THE CONCEPT

`git status` prints a full report of exactly what state your working directory and staging area are in right now: which files are untracked, which are modified but unstaged, and which are staged and ready to commit.

WHAT GIT DOES INTERNALLY

Git compares three things against each other to build this report: the files currently on your disk, the snapshot recorded in the staging area (index), and the snapshot recorded in your last commit. Any differences between these three are what `git status` summarises for you in plain English.

SYNTAX

`git status`

WHEN TO USE THIS

Run this constantly — genuinely, before and after almost every other Git command you run, especially while you are still learning. It never changes anything; it only reports.

COMMON MISTAKES

- Skipping `git status` and guessing what will happen when you commit or push — leading to accidentally committing the wrong files.
- Not reading the suggested next-step text Git prints, which almost always tells you exactly which command to run next.

Seeing All Four States in Action

Here is a single terminal session showing a file moving through every state.

```
terminal

# 1. A brand new file: UNTRACKED
$ touch utils.py
$ git status
Untracked files:
  utils.py

# 2. Stage it: STAGED
$ git add utils.py
$ git status
Changes to be committed:
  new file:   utils.py

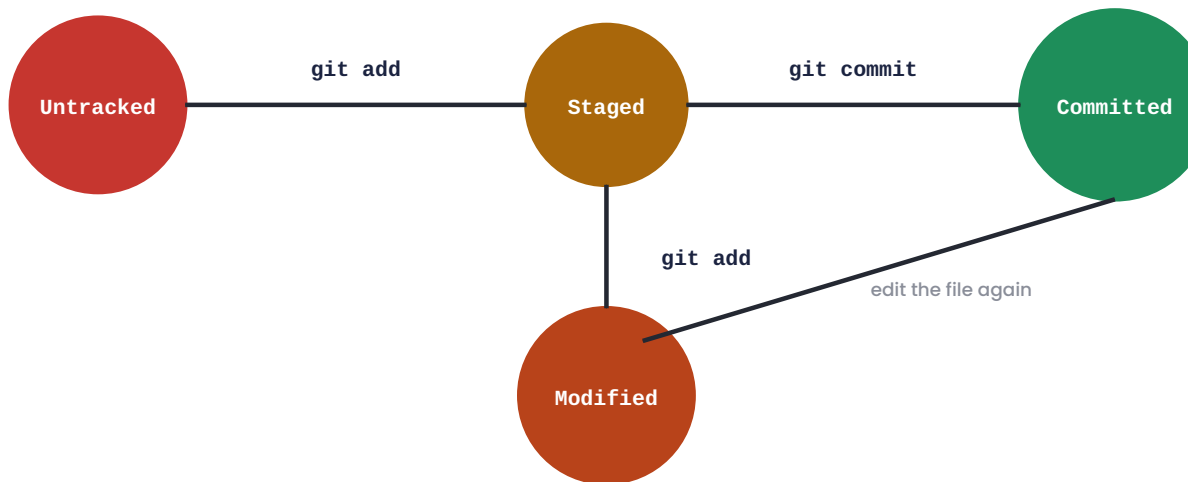
# 3. Commit it, then edit it again: MODIFIED
$ git commit -m "Add utils module"
$ echo "# TODO: add helper functions" >> utils.py
$ git status
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  modified:   utils.py

# 4. Nothing left to save: CLEAN
$ git add utils.py
$ git commit -m "Add TODO note"
$ git status
On branch main
nothing to commit, working tree clean
```

WHEN TO USE THIS

The phrase **"working tree clean"** is Git's way of telling you that absolutely everything currently on disk matches your last commit exactly. It is one of the most reassuring sentences in all of software development.

A State-Transition Map



Once a file has been committed at least once, editing it again moves it to Modified, from which `git add` stages it once more.

COMMON MISTAKES

- Assuming a file is safely committed just because it's been "added" — staged and committed are different states.
- Not noticing when `git status` lists a file under both "Changes to be committed" and "Changes not staged" at once — this means you staged it, then edited it again afterward.

MINI PRACTICE TASK

In any repository, practice reading state:

1. Create a new file and run `git status` — confirm it's Untracked.
2. Stage it and run `git status` again — confirm it's Staged.
3. Commit it, edit it again, and run `git status` — confirm it's now Modified.
4. Stage and commit once more until you see "working tree clean."

git add

Choosing exactly what belongs in your next commit

Priya has changed three files but only wants to save one of them right now. This is exactly the kind of control `git add` gives you.

THE STORY CONTINUES

Priya is working on the login feature. While testing it, she has also accidentally left some debug `print()` statements in a completely unrelated file, `reports.py`. She wants to save her login progress without accidentally saving those debug lines too.

```
git add <file>
```

THE PROBLEM

If committing simply saved "everything that changed" with no way to choose, Priya would be forced to either commit her debug code by accident, or manually undo her debug changes before she could safely commit anything at all.

THE CONCEPT

`git add` lets you choose exactly which changes move into the staging area — and therefore exactly which changes will be included the next time you run `git commit`. Anything you don't add is simply left out, untouched, ready to be committed later.

WHAT GIT DOES INTERNALLY

When you stage a file, Git takes a snapshot of that file's *exact current content* and records it in the index. If you edit the file again after staging it, the newer changes are not automatically included — you would need to run `git add` again to update the staged snapshot.

SYNTAX

```
git add <file>
```

terminal

```
$ git add login.py
$ git status
Changes to be committed:
  modified:   login.py

Changes not staged for commit:
  modified:   reports.py
```

WHEN TO USE THIS

Use the single-file form whenever you want fine-grained control over exactly what enters your next commit — which, as a habit, makes for a much cleaner, more readable project history.

COMMON MISTAKES

- Assuming `git add` uploads the file anywhere — it is purely local (see Chapter 3).

Staging Everything at Once

```
git add .
```

Stages *every* new and modified file in the current folder and all its subfolders in one go. The dot means "this directory."

SYNTAX

```
git add .
```

```
terminal
$ git add .
$ git status
Changes to be committed:
  modified:   login.py
  modified:   reports.py
  new file:   session.py
```

When to use it: When you've reviewed every change with `git status` / `git diff` first, and genuinely want everything included.

Watch out: Running `git add .` as a reflex, without checking what it will actually stage — this is exactly how Priya's debug `print()` statements could sneak into a commit.

DANGER ZONE

Blind `git add .` is one of the most common sources of messy, embarrassing commits. It can accidentally stage debug code, temporary files, large binaries, or — worst of all — secret API keys and passwords sitting in a config file. Always run `git status` first to see what `git add .` is about to pick up. Chapter 10 shows you how to permanently exclude files like this using `.gitignore`.

Staging Specific Files or Patterns

```
git add file1 file2 file3
```

Stages several specific, named files in a single command, separated by spaces — useful when you want more than one file but not literally everything.

SYNTAX

```
git add login.py session.py
```

```
terminal
$ git add login.py session.py
```

git add *.java

Uses a **wildcard pattern** to stage every file matching that pattern — here, every file ending in `.java` in the current folder.

SYNTAX

```
git add *.java
```

terminal

```
$ git add *.java
add 'Attendance.java'
add 'Student.java'
```

When to use it: When you've changed many files of the same type and want to stage them together without typing every name.

Watch out: Forgetting that a wildcard like `*.java` only matches the current folder by default, not subfolders — use `git add .` or a more specific pattern for nested files.

Selective Staging: Why It Matters

THE CONCEPT

The ability to stage only *some* changes is called **selective staging**, and it is one of Git's most powerful, most under-appreciated features. It lets you split one messy work session into several small, focused, readable commits — for example, one commit for the login feature and a completely separate commit later for cleaning up debug code — instead of one giant commit that mixes unrelated changes together.

COMMON MISTAKES

- Using `git add .` without first checking `git status`.
- Believing staging a file "locks" its content forever — editing it again after staging simply makes it Modified again, and you'll need to re-run `git add`.
- Trying to stage a file that doesn't exist yet (a typo in the filename) and being confused by the "pathspec did not match any files" error.

MINI PRACTICE TASK

Practice selective staging:

1. Edit two different files in a repository.
2. Stage only one of them using `git add` with its exact filename.
3. Run `git status` and confirm exactly one file shows as staged, and the other still shows as modified.

git commit

Saving a snapshot your future self will thank you for

Rahul is about to type the laziest commit message in the history of the team's project. Priya stops him just in time.

THE STORY CONTINUES

Rahul finishes a bug fix and, without thinking, types `git commit -m "changes"`. Priya glances over. "Two weeks from now, when something breaks, that message will tell you absolutely nothing," she says. "A commit message is a note to your future self — and to anyone else who has to figure out what you did, and why."

```
git commit -m "message"
```

THE PROBLEM

Staging changes with `git add` only prepares them. Nothing is permanently saved to history until a commit actually happens — and every commit needs a clear record of what changed and why.

THE CONCEPT

A **commit** is a permanent, named snapshot of your entire staged project at one point in time. Unlike a vague "save," every commit carries an author, a timestamp, a unique ID, and a message explaining the change — forming a readable timeline of your whole project's history.

WHAT GIT DOES INTERNALLY

When you commit, Git takes everything currently in the staging area, packages it as a new commit object, and links that new object to the commit that came directly before it (its "parent"). This parent link is what allows Git to reconstruct the entire project history as a chain, one commit at a time, all the way back to the very first commit.

SYNTAX

```
git commit -m "A short, clear description of the change"
```

 terminal

```
$ git commit -m "Fix incorrect attendance percentage rounding"
[main 8b3e1af] Fix incorrect attendance percentage rounding
1 file changed, 3 insertions(+), 1 deletion(-)
```

Part of the command

What it means

`git commit`

Save everything currently staged as a new, permanent snapshot.

`-m`

Short for "message" — lets you supply the commit message directly on the command line.

`"..."`

The message itself, in quotes so spaces are treated as one argument.

WHEN TO USE THIS

Every time you finish one logical, complete unit of work — a bug fix, a small feature, a refactor — not once at the very end of a whole day's work.

COMMON MISTAKES

- Writing vague messages like `"changes"`, `"update"`, or `"fix"` that give future-you zero useful information.
- Bundling several unrelated changes into one giant commit, making it impossible to undo just one of them later.
- Forgetting the quotes, which causes Git to interpret only the first word as the message.

Good vs. Bad Commit Messages

GOOD

```
git commit -m "Add login input validation"

git commit -m "Fix null pointer in
attendance report"

git commit -m "Refactor percentage
calculation for readability"
```

BAD

```
git commit -m "changes"

git commit -m "fix"

git commit -m "asdf"
```

THE CONCEPT

A good commit message is written in the **imperative mood** (as if giving a command: "Add," "Fix," "Remove," not "Added" or "Fixes"), states *what* changed, and is specific enough that someone skimming your project history months later understands the change without opening the code.

Atomic Commits

THE CONCEPT

An **atomic commit** is a commit that contains exactly one logical change — nothing more, nothing less. Atomic commits are easy to read, easy to undo individually (Chapter 9), and easy to review in a Pull Request (Chapter 18). If describing your commit needs the word "and" — "fixed login and updated styling and removed a file" — it should probably have been two or three separate commits.

Commit IDs (Hashes)

WHAT GIT DOES INTERNALLY

Every commit is identified by a **SHA-1 hash** — a 40-character fingerprint (usually shown shortened to 7 characters, like `8b3e1af`) calculated from the commit's content, author, timestamp, and parent commit. Because it's derived from the content itself, even a one-character difference produces a completely different hash — which is what lets Git detect, reference, and verify every single commit with total precision.

Committing Without -m

```
git commit
```

Running `git commit` with no message opens your default text editor (often Vim) so you can write a longer, multi-line commit message — useful for a commit that needs more explanation than one short line.

SYNTAX

```
git commit
```

```
terminal
```

```
$ git commit
# opens an editor – type your message, save, and close it
```

When to use it: When a change is complex enough to deserve a title line plus a longer explanation paragraph below it.

Watch out: Not knowing how to exit the editor. In Vim: press `Esc`, then type `:wq` and press Enter to save and quit.

Fixing Your Last Commit

```
git commit --amend
```

THE STORY CONTINUES

Two minutes after committing, Rahul notices a typo in his commit message – and also realises he forgot to stage one small file that belonged in that same commit.

THE CONCEPT

`git commit --amend` replaces your most recent commit with a new one, letting you fix the message, add forgotten staged changes, or both – instead of creating a separate, messy "oops, forgot a file" commit right after.

SYNTAX

```
git commit --amend -m "Corrected message"
```

```
terminal
```

```
$ git add forgotten_file.py
$ git commit --amend -m "Fix attendance percentage rounding"
[main 4c7a2de] Fix attendance percentage rounding
Date: Tue Sep 15 10:02:11 2026
2 files changed, 5 insertions(+), 1 deletion(-)
```

WHEN TO USE THIS

Only on a commit you have **not yet pushed** to GitHub. Amending replaces the old commit with a brand-new one that has a different hash entirely.

COMMON MISTAKES

- Amending a commit that has already been pushed and that teammates may have already pulled – this rewrites history and can cause serious confusion for anyone else who already has the old version (see Chapter 24 on rebase for the full explanation of why rewriting shared history is risky).

MINI PRACTICE TASK

Practice the full commit workflow:

1. Make a small change and commit it with a clear, imperative-mood message.
2. Make one more tiny change (like fixing a typo in that same file) and use `git commit --amend` to fold it into the previous commit instead of creating a new one.
3. Run `git log --oneline` and confirm you still see only one commit for both changes.

Viewing History

Reading the story your commits have already written

Weeks into the project, Arjun needs to know exactly when a bug was introduced. The answer is sitting in his commit history the whole time.

THE STORY CONTINUES

The attendance percentage has started showing negative numbers for some students, and nobody remembers when it last worked correctly. "You don't have to remember," Kabir says. "Git already remembers everything. You just have to know how to ask it."

git log

THE CONCEPT

`git log` displays the complete commit history of the current branch, most recent commit first: full hash, author, date, and message for every single commit ever made.

SYNTAX

`git log`

```
terminal

$ git log
commit 8b3e1af9c2d4e5f6a7b8c9d0e1f2a3b4c5d6e7f8 (HEAD -> main)
Author: Arjun Menon <arjun.menon@example.com>
Date: Tue Sep 15 09:40:02 2026 +0530

    Fix attendance percentage rounding

commit 3f2a91c7b1a0d9e8f7c6b5a4938271605f4e3d2c
Author: Arjun Menon <arjun.menon@example.com>
Date: Mon Sep 14 21:15:47 2026 +0530

    Initial commit
```

WHEN TO USE THIS

Any time you need to investigate the past: who changed what, when, and why.

COMMON MISTAKES

- Trying to scroll a long `git log` output and not knowing how to exit — press `q` to quit the paginated view.

Friendlier Views of History

git log --oneline

Condenses each commit down to one line: a short hash and the message. This is the view you will reach for most often

once a project has more than a handful of commits.

SYNTAX

```
git log --oneline
```

terminal

```
$ git log --oneline
8b3e1af Fix attendance percentage rounding
3f2a91c Initial commit
```

```
git log --graph
```

Draws an ASCII graph alongside the log, showing how branches diverged and merged — essential once you have more than one branch (Chapter 15 onward).

SYNTAX

```
git log --graph --oneline --all
```

terminal

```
$ git log --graph --oneline --all
* 8b3e1af (HEAD -> main) Fix attendance percentage rounding
| * a91cd3e (feature-login) Add login validation
|/
* 3f2a91c Initial commit
```

```
git log --all
```

Shows commits from *every* branch, not just the one you currently have checked out — often combined with `--graph` and `--oneline` as shown above.

SYNTAX

```
git log --all
```

```
git log --decorate
```

Explicitly labels each commit with the branch and tag names that currently point to it. (This is actually turned on by default in most modern Git installs — you'll see labels like `(HEAD -> main)` already, as in the examples above.)

SYNTAX

```
git log --decorate
```

```
git show <commit>
```

Displays full details of one specific commit — its metadata *and* the exact code changes (diff) it introduced.

SYNTAX

```
git show 8b3e1af
```

```
terminal

$ git show 8b3e1af
commit 8b3e1af9c2d4e5f6a7b8c9d0e1f2a3b4c5d6e7f8
Author: Arjun Menon <arjun.menon@example.com>

    Fix attendance percentage rounding
```

When to use it: When you've found the commit you're interested in (often via `git log`) and want to see exactly what code it changed.

git diff: Comparing Versions

git diff

THE STORY CONTINUES

Before staging anything, Arjun wants to double-check exactly what he changed in `attendance.py` since his last commit.

THE CONCEPT

`git diff`, run with no arguments, shows the line-by-line differences between your working directory and the staging area — in other words, exactly what you've changed but *haven't* staged yet.

SYNTAX

`git diff`

WHEN TO USE THIS

Right before staging, as a final review of exactly what you're about to add.

COMMON MISTAKES

- Confusing plain `git diff` (unstaged changes) with `git diff --staged` (staged changes) — they show two different things.

```
diff --git a/attendance.py b/attendance.py
def calculate_percentage(present, total):
-     return present / total
+     return round((present / total) * 100, 2)
```

Command	What it compares
<code>git diff</code>	Working directory vs. Staging area (your unstaged edits)
<code>git diff --staged</code>	Staging area vs. Last commit (what you're about to commit)
<code>git diff HEAD</code>	Working directory vs. Last commit (all changes, staged or not, combined)

COMMON MISTAKES

- Reading a long `git log` and not knowing `q` quits the paginated view.
- Assuming `git diff` shows staged changes too – use `git diff --staged` (also called `git diff --cached`) for that.

MINI PRACTICE TASK

Explore your own project's history:

1. Make a few commits to any repository, then run `git log` and `git log --oneline`.
2. Pick one commit's short hash and run `git show` on it.
3. Edit a tracked file without staging it, and compare the output of `git diff` before and after running `git add`.

Undoing Changes

Five different kinds of ‘undo’, and how to pick the right one

Something has gone wrong. Before you panic, this chapter is about the one thing every beginner desperately needs: a safe way back.

THE STORY CONTINUES

Rahul has just broken the login feature while "quickly trying something." He wants to undo it — but he's not sure if he wants to undo an edit he hasn't saved yet, un-stage something, undo an entire commit, or go back several commits at once. "Those are four completely different problems," Kabir says, "and Git has a precise tool for each one. Using the wrong one is how people accidentally lose "work — so let's go slowly."

Four Different Kinds of ‘Undo’

Situation	What you actually want	The right tool
I edited a file and want to throw away the edit	Discard an unstaged change in the working directory	<code>git restore</code>
I staged a file but changed my mind	Move it back to unstaged, keeping the edit itself	<code>git restore --staged</code>
I already committed, and want a NEW commit that undoes it (safe for shared history)	Add an inverse commit	<code>git revert</code>
I want to rewind history itself to an earlier commit (only before pushing / sharing)	Move HEAD and the branch pointer backward	<code>git reset</code>

`git restore <file>`

THE PROBLEM

Rahul's broken edit to `login.py` hasn't been staged or committed — it exists only in his working directory. He wants that file to look exactly like it did at the last commit, as if he'd never touched it.

THE CONCEPT

`git restore <file>` discards uncommitted changes in your working directory for that file, replacing it with the version from your last commit.

SYNTAX

`git restore login.py`

```

terminal
$ git status
    modified:   login.py
$ git restore login.py
$ git status
nothing to commit, working tree clean

```

WHEN TO USE THIS

When you've made changes you have not staged, and you're certain you want to throw them away completely.

COMMON MISTAKES

- Running this and expecting to get the discarded code back afterward — you cannot. Unstaged, uncommitted changes discarded with `git restore` are gone for good.

`git restore .`

The same idea as above, but applied to *every* modified file in the current folder at once, instead of naming one file.

SYNTAX

```
git restore .
```

Watch out: Running this in a hurry and discarding changes you actually wanted to keep in a different file you forgot about — always run `git status` first.

`git restore --staged <file>`

THE STORY CONTINUES

Priya stages a file with `git add`, then realises she isn't ready to commit it yet after all. She doesn't want to lose her edits — just move the file back to "unstaged."

THE CONCEPT

`git restore --staged <file>` unstages a file — it moves back to the "modified" state, but every edit you made is fully preserved in the working directory.

SYNTAX

```
git restore --staged login.py
```

```
terminal

$ git restore --staged login.py
$ git status
Changes not staged for commit:
  modified:   login.py
```

WHEN TO USE THIS

When you've staged something too early, or staged the wrong file, but still want to keep the actual edits.

COMMON MISTAKES

- Confusing this with `git restore` (no `--staged`) — the flag is the difference between unstaging safely and discarding permanently.

NOTE

You may also see the older, equivalent command `git reset HEAD <file>` in tutorials and Stack Overflow answers — it does the same job as `git restore --staged`. `git restore` was introduced later specifically to make "undo" commands less confusing, so this book teaches it as the modern default.

Undoing a Commit You've Already Made

```
git revert <commit>
```

THE STORY CONTINUES

The team already pushed a commit to GitHub that turns out to break the dashboard. Meena has already pulled that commit onto her own laptop. Simply erasing the commit from history now would be dangerous — it could desync everyone's history.

THE CONCEPT

`git revert` creates a **brand-new commit** that applies the exact opposite changes of the commit you're reverting, effectively undoing its effect — without deleting or rewriting any history. The broken commit still exists in the log; a new commit right after it simply cancels it out.

SYNTAX

```
git revert <commit-hash>
```

terminal

```
$ git revert 8b3e1af
[main a5f9d02] Revert "Fix attendance percentage rounding"
1 file changed, 1 insertion(+), 3 deletions(-)
```

WHEN TO USE THIS

Whenever the commit you're undoing has **already been pushed** and might already be on a teammate's machine. Reverting is the safe, collaboration-friendly way to undo.

COMMON MISTAKES

- Reaching for `git reset` instead on a commit that's already shared with others — that rewrites history teammates already have, and can cause serious sync problems (see below).

Rewinding History: `git reset`

DANGER ZONE

`git reset` is one of the few Git commands that can genuinely and permanently destroy work if used carelessly. Read this section slowly.

THE CONCEPT

`git reset HEAD~1` moves your current branch pointer backward by one commit — as if that last commit had never happened. Exactly *what* happens to the changes from that undone commit depends entirely on which flag you use: `--soft`, `--mixed`, or `--hard`.

`--soft`

Commit undone
Staging: kept
Working dir: kept

Safest — nothing lost,
changes stay staged

`--mixed (default)`

Commit undone
Staging: cleared
Working dir: kept

Changes survive as
unstaged edits

`--hard`

Commit undone
Staging: cleared
Working dir: ERASED

**DANGER: changes are
permanently deleted**

What happens to your undone commit's changes under each reset mode.

`git reset --soft HEAD~1`

Undoes the last commit, but keeps all its changes fully staged, ready to be re-committed (perhaps with a better message, or combined with more changes).

SYNTAX

```
git reset --soft HEAD~1
```

`git reset --mixed HEAD~1`

The default mode if you omit the flag entirely. Undoes the last commit and unstages its changes, but keeps them safely sitting in your working directory as edits.

SYNTAX

```
git reset --mixed HEAD~1    (same as: git reset HEAD~1)
```

`git reset --hard HEAD~1`

THE CONCEPT

Undoes the last commit **and permanently deletes** the changes it contained from your working directory too. After this command, it is as if that work was never done at all.

SYNTAX

```
git reset --hard HEAD~1
```

terminal

```
$ git reset --hard HEAD~1
HEAD is now at 3f2a91c Initial commit
```

WHEN TO USE THIS

Only when you are **completely certain** you want to throw away both the commit and its actual code changes, and that commit has never been pushed anywhere.

COMMON MISTAKES

- Running `--hard` out of habit because it 'sounds thorough' — it is the single most dangerous everyday Git command for beginners.
- Running `--hard` on a commit that has already been pushed and pulled by teammates, causing painful history mismatches.

DANGER ZONE

There is no undo for `git reset --hard`. Any uncommitted work in your working directory at the moment you run it is deleted immediately and irreversibly. Always run `git status` first, and when in doubt, commit or stash (Chapter 23) your work before resetting anything.

git revert vs. git reset: Which One Should You Use?

	git revert	git reset
Changes history?	No — adds a new commit	Yes — moves the branch pointer backward, removing commits from the branch's history
Safe for pushed/shared commits?	Yes, always	No — only safe on commits nobody else has pulled yet
Good default choice for teams	Yes	Only for your own, still-local, still-private commits

COMMON MISTAKES

- Using `git reset --hard` as a general-purpose "undo" without understanding it deletes uncommitted work.
- Using `git reset` on commits already pushed to a shared branch, instead of the collaboration-safe `git revert`.
- Confusing `git restore` (working directory / staging area, uncommitted changes) with `git reset` (moves which commit your branch points to).

MINI PRACTICE TASK

Practice safely, on a throwaway test repository:

1. Make three small commits in a row.
2. Run `git reset --soft HEAD~1` and confirm the last commit's changes are now staged.
3. Commit again, then try `git reset --mixed HEAD~1` and check `git status`.
4. Only once you're comfortable, try `git reset --hard HEAD~1` on a commit you don't need, and confirm the change is really gone.

.gitignore

Teaching Git what to politely ignore

Rahul's repository suddenly triples in size after he installs some packages. None of that belongs in Git history at all.

THE STORY CONTINUES

Rahul runs his Node.js backend for the first time, which downloads hundreds of library files into a folder called `node_modules`. When he runs `git status`, Git wants to track thousands of new files. "None of that should ever go into your repository," Kabir says. "Some files should always stay out of version control – and you tell Git exactly which ones with a file called `.gitignore`."

Why Some Files Should Never Be Committed

THE CONCEPT

Not every file in a project folder belongs in its history. Three categories come up constantly:

Category	Examples	Why exclude it
Regenerable dependency files	<code>node_modules/</code> , <code>target/</code> , <code>build/</code>	Huge, and can always be recreated from a config file (like <code>package.json</code>) – storing them just bloats the repository.
Machine/editor-specific files	<code>.idea/</code> , <code>.vscode/</code> , <code>*.class</code>	Specific to one person's computer or editor setup, and meaningless to anyone else.
Secrets and environment config	<code>.env</code> , API keys, passwords	Committing these exposes sensitive credentials to anyone who can see the repository – including, on a public GitHub repo, the entire internet.

DANGER ZONE

Never commit secrets, passwords, or API keys – not even to a private repository. Once a secret is committed, it exists permanently in your Git history, in every clone of the repository, even if you delete the file in a later commit. If a real secret is ever committed by mistake, the correct fix is to treat that secret as compromised and generate a brand-new one, not just delete the file.

.gitignore

THE CONCEPT

A file named exactly `.gitignore`, placed in your project's root folder, lists file and folder patterns that Git should never track, stage, or warn you about – even if they physically exist inside the project folder.

SYNTAX

```
# Create it like any other text file
touch .gitignore
```

```
terminal

$ echo 'node_modules/' >> .gitignore
$ echo '.env' >> .gitignore
$ cat .gitignore
node_modules/
.env
$ git status
Untracked files:
  .gitignore
# node_modules/ and .env no longer appear at all
```

WHEN TO USE THIS

Create a `.gitignore` file at the very start of every project, before your first commit, so nothing unwanted is ever accidentally tracked in the first place.

COMMON MISTAKES

- Adding a file to `.gitignore` after it has already been committed — “ignore” only prevents future tracking. An already-tracked file needs to be explicitly removed with `git rm --cached <file>` before it will actually disappear from future commits.
- Believing `.gitignore` deletes or hides files from your computer — it does not. The files still exist; Git simply stops paying attention to them.

.gitignore by Project Type

Stack	Typical .gitignore entries
Java	<code>*.class</code> <code>target/</code> <code>*.jar</code>
Python	<code>__pycache__/</code> <code>*.pyc</code> <code>venv/</code> <code>.env</code>
Node.js	<code>node_modules/</code> <code>.env</code> <code>npm-debug.log</code>
Android / Gradle	<code>build/</code> <code>.gradle/</code> <code>local.properties</code> <code>*.apk</code>

TIP

GitHub can generate a solid starter `.gitignore` for your exact language or framework automatically when you create a new repository on the website — you rarely need to write one entirely from scratch.

COMMON MISTAKES

- Writing a `.gitignore` only after already committing hundreds of dependency files.
- Ignoring `.env.example` along with `.env` by mistake — the `.example` version usually contains no real secrets and is meant to be shared, showing teammates which variables they need to set.

MINI PRACTICE TASK

Set up a `.gitignore` of your own:

1. In any project, create a `.gitignore` file listing at least one folder and one file pattern relevant to your stack.
2. Create a matching throwaway file or folder and confirm `git status` no longer lists it.
3. If you've already accidentally committed something that should be ignored, look up and try `git rm --cached` to remove it from tracking without deleting it from disk.

PART III

GOING REMOTE WITH GITHUB

Connecting Git to GitHub

Giving your local repository a home on the internet

Arjun's project has a perfect local history — and zero backup. One spilled cup of chai near his laptop, and all of it could vanish.

THE STORY CONTINUES

"Your commits are safe from mistakes," Kabir says, "but they're not safe from a stolen laptop, a dead hard drive, or a coffee spill. Everything currently lives in one single `.git` folder, in one single place. It's time to give this project a home on GitHub too."

Step 1: Create a GitHub Account

THE CONCEPT

Visit github.com and sign up with an email address, a username, and a password (GitHub will also prompt you to set up two-factor authentication, which is strongly recommended). This account is separate from Git itself — it is what lets you own repositories, open Pull Requests, and collaborate on GitHub's website.

Step 2: Create a Repository on GitHub

THE CONCEPT

1. Click the **+** icon in the top right of any GitHub page, then **New repository**.
2. Give it a name, for example `student-attendance-system`.
3. Choose **Public** (anyone can see it) or **Private** (only people you invite can see it).
4. Since Arjun already has a local project with commits, he leaves the "Initialize with README" option **unchecked** — otherwise GitHub would create a history that conflicts with his own.
5. Click **Create repository**. GitHub shows a page of setup commands — the exact ones covered next.

Step 3: Connect Your Local Repository

```
git remote add origin <URL>
```

THE CONCEPT

A **remote** is simply a nickname Git uses for a URL pointing to another copy of the repository — almost always one hosted on GitHub. This command teaches your local repository that address, under a name you choose.

WHAT GIT DOES INTERNALLY

Git stores this remote name and URL in your project's local `.git/config` file. From now on, wherever a Git command asks for a remote, you can type its short nickname instead of the full URL.

SYNTAX

```
git remote add <name> <url>
```

terminal

```
$ git remote add origin https://github.com/arjunmenon/student-attendance-system.git
$ git remote -v
origin https://github.com/arjunmenon/student-attendance-system.git (fetch)
origin https://github.com/arjunmenon/student-attendance-system.git (push)
```

Part of the command	What it means
---------------------	---------------

<code>git remote add</code>	Register a new remote connection.
-----------------------------	-----------------------------------

<code>origin</code>	The conventional nickname for your main remote. You could technically name it anything, but almost every project calls its primary remote “origin.”
---------------------	---

The web address of the repository on GitHub, copied from GitHub's “Quick setup” page.

WHEN TO USE THIS

Exactly once per project, right after creating the matching repository on GitHub.

COMMON MISTAKES

- Running this command twice, which produces a `remote origin already exists` error (covered fully in Chapter 30).
- Copying the wrong URL — double check it matches the repository you just created.

git branch -M main

Renames your current branch to `main`, guaranteeing it matches the branch name GitHub expects by default (relevant mainly if your local default branch ended up named something else, like the historical `master`).

SYNTAX

```
git branch -M main
```

git push -u origin main

THE STORY CONTINUES

With the remote registered, Arjun is ready to upload his entire commit history to GitHub for the very first time.

THE CONCEPT

This command uploads your local `main` branch — and every commit on it — to the `origin` remote, and additionally sets up a permanent link (“upstream tracking”) between your local `main` and GitHub’s `main`, so future pushes and pulls can simply be typed as `git push` or `git pull`, with no extra arguments.

SYNTAX

```
git push -u origin main
```

```
terminal

$ git push -u origin main
Enumerating objects: 6, done.
Writing objects: 100% (6/6), 512 bytes | 512.00 KiB/s, done.
To https://github.com/arjunmenon/student-attendance-system.git
 * [new branch]      main -> main
Branch 'main' set up to track remote branch 'main' from 'origin'.
```

Part of the command	What it means
<code>git push</code>	Upload local commits to a remote repository.
<code>-u</code>	Short for "--set-upstream" — remembers this remote+branch pairing for next time.
<code>origin</code>	Which remote to push to.
<code>main</code>	Which branch to push.

WHEN TO USE THIS

The very first time you push a given branch. After this, plain `git push` is enough for that branch.

COMMON MISTAKES

- Forgetting `-u` on the first push, then being confused why later `git push` (with no arguments) fails with an error about no upstream branch.

HTTPS vs. SSH: Two Ways to Authenticate

THE CONCEPT

When Git talks to GitHub, it needs to prove *who you are* — GitHub won't accept a push from just anyone. There are two common ways to do this.

	HTTPS	SSH
URL looks like	<code>https://github.com/user/repo.git</code>	<code>git@github.com:user/repo.git</code>
How you authenticate	A Personal Access Token (not your GitHub password — GitHub no longer accepts plain passwords over HTTPS)	An SSH key pair set up once on your computer
Best for	Beginners, and machines you don't use every day	Frequent use on your own personal machine, where typing a token repeatedly gets tedious

```
git remote add origin git@github.com:username/repository.git
```

The same `remote add` command as before, but using an SSH-style URL instead of an HTTPS one — this only works once an SSH key has been generated and added to your GitHub account (Chapter 28 covers this in full).

SYNTAX

```
git remote add origin git@github.com:username/repository.git
```

NOTE

Full, step-by-step SSH key setup is covered in Chapter 28 (GitHub Authentication). For now, know that both HTTPS and SSH accomplish exactly the same thing — uploading and downloading code — they simply prove your identity differently.

COMMON MISTAKES

- Checking "Initialize this repository with a README" on GitHub when you already have a local project with commits, creating two conflicting histories that need to be reconciled before the first push.
- Trying to push before creating the remote repository on GitHub at all.
- Mixing up which direction things move — `git remote add` only registers an address; it doesn't transfer any code by itself.

MINI PRACTICE TASK

Publish a project of your own to GitHub:

1. Create a new, empty repository on GitHub (don't initialize it with a README).
2. In a local project with at least one commit, run `git remote add origin <url>` using that repository's URL.
3. Run `git push -u origin main` and refresh the GitHub page to see your files appear.

CHAPTER 12

git push

Sending your local commits out into the world

Now that `origin` is connected, `push` becomes part of Arjun's everyday rhythm.

THE STORY CONTINUES

With the remote set up, Arjun makes a small fix, commits it locally, and now needs to get that one new commit onto GitHub too – without repeating the full setup from Chapter 11.

git push

THE CONCEPT

`git push`, run with no arguments, uploads any commits on your current branch that the remote doesn't have yet – as long as that branch already has an upstream tracking connection set up (via `-u`, as in Chapter 11).

WHAT GIT DOES INTERNALLY

Git compares your local branch's commit history against the remote branch's history. Any commits that exist locally but not on the remote are uploaded, and the remote branch pointer is moved forward to match.

SYNTAX

`git push`

terminal

```
$ git push
Enumerating objects: 4, done.
To https://github.com/arjunmenon/student-attendance-system.git
  3f2a91c..8b3e1af  main -> main
```

WHEN TO USE THIS

Any time you've made one or more local commits you want to back up or share, on a branch that already tracks a remote.

COMMON MISTAKES

- Trying to push with nothing committed yet – remember, only *commits* can be pushed, not staged or unstaged changes.
- Expecting `git push` to work before ever running `git push -u` once, on a brand new branch.

Pushing a Specific Branch

git push origin main

The fully explicit form: push the local `main` branch to the remote named `origin`. Functionally identical to plain `git push` once upstream tracking is set up, but useful to write out explicitly, especially in scripts or when working with more

than one remote.

SYNTAX

```
git push origin main
```

```
git push -u origin main
```

THE CONCEPT

As introduced in Chapter 11, adding `-u` the first time you push a branch sets up permanent tracking, so every future push from that branch can simply be `git push`.

SYNTAX

```
git push -u origin main
```

```
git push origin feature-login
```

Pushes a branch *other* than your current default — here, a feature branch called `feature-login` — to the remote, usually to open a Pull Request from it (Chapter 18).

SYNTAX

```
git push origin feature-login
```

terminal

```
$ git push origin feature-login
remote: Create a pull request for 'feature-login' on GitHub by visiting:
remote:   https://github.com/arjunmenon/student-attendance-system/pull/new/feature-login
```

When to use it: Whenever you're working on a branch other than main and want to share it or open a PR.

Local vs. Remote, and Why a Commit Must Exist First

THE CONCEPT

`git push` only ever sends **commits** — permanently saved snapshots. It has no concept of "uncommitted work" at all. If you haven't committed, there is nothing for `git push` to send, no matter how many files you've edited or staged.

COMMON MISTAKES

- Trying to push edits that were only staged, not committed.
- Being surprised by a `rejected - non-fast-forward` error when the remote has commits your local branch doesn't — the fix (`git pull` first) is covered fully in Chapters 14 and 30.

MINI PRACTICE TASK

Practice the push rhythm:

1. Make a small edit to a tracked file, commit it, and push using plain `git push`.
2. Create a new branch (a quick preview of Chapter 15: `git switch -c test-branch`), commit something on it, and push it explicitly with `git push origin test-branch`.

git clone

Downloading a complete project, history included

Meena joins the team weeks into the project. She needs everything Arjun, Priya, and Rahul already have — instantly.

THE STORY CONTINUES

Meena has just joined the College Event Management System project (you'll meet this full team project in Chapter 26). The project already exists on GitHub, with dozens of commits. She doesn't want to start from nothing and manually recreate it — she wants an exact, complete copy on her own laptop.

```
git clone <repository-url>
```

THE PROBLEM

Meena has no local copy of the project and no `.git` history at all. `git init` wouldn't help here — it only creates an empty repository, and she needs all of the team's existing work.

THE CONCEPT

`git clone` downloads a complete copy of a repository from a remote URL: every file, every branch, and the entire commit history, from the very first commit onward. It also automatically sets up a remote named `origin` pointing back to where it was cloned from.

WHAT GIT DOES INTERNALLY

Because Git is distributed (Chapter 1), cloning doesn't just grab the latest snapshot of files — it downloads the full `.git` object database. The moment the clone finishes, Meena's laptop holds a complete, independent backup of the project's entire history, exactly as capable as anyone else's copy.

SYNTAX

```
git clone <repository-url>
```

```
terminal
$ git clone https://github.com/college-team/event-management-system.git
Cloning into 'event-management-system'...
remote: Enumerating objects: 142, done.
Receiving objects: 100% (142/142), 38.20 KiB | 2.30 MiB/s, done.
Resolving deltas: 100% (54/54), done.
```

Part of the command

What it means

`git clone`

Download a complete copy of a remote repository.

The address of the repository, copied from GitHub's green "Code" button.

WHEN TO USE THIS

Whenever a repository already exists on GitHub (or elsewhere) and you need a local copy — joining an existing team project, or contributing to an open-source project (Chapter 19).

COMMON MISTAKES

- Running `git init` and then manually trying to recreate an existing project's history by hand, instead of simply cloning it.
- Cloning into a folder that already exists and already has content, which can cause Git to refuse or create a confusing nested folder.

```
git clone <repository-url> project-name
```

Adding a second argument clones into a folder with a name of your choosing, instead of the default (which matches the repository's name on GitHub).

SYNTAX

```
git clone https://github.com/college-team/event-management-system.git ems
```

terminal

```
$ git clone https://github.com/college-team/event-management-system.git ems
Cloning into 'ems'...
```

When to use it: When the default folder name is inconvenient, or you already have a folder with that name.

clone vs. init: Two Different Starting Points

	git init	git clone
Starting point	A brand-new, empty project	A project that already exists somewhere else (usually GitHub)
History after running it	None yet — zero commits	The complete, existing history — potentially hundreds of commits
Remote set up automatically?	No — you add one manually (Chapter 11)	Yes — <code>origin</code> is configured automatically

COMMON MISTAKES

- Using `git init` when you actually needed `git clone` (joining an existing project), and ending up with an empty repository disconnected from the team's real history.
- Forgetting to `cd` into the newly cloned folder before running further Git commands.

MINI PRACTICE TASK

Practice cloning:

1. Find any public repository on GitHub (even one of your own from an earlier chapter).
2. Copy its HTTPS URL and run `git clone` on it in a different folder.
3. Run `git log --oneline` inside the cloned folder and confirm you see its full history.
4. Run `git remote -v` and confirm `origin` was configured automatically.

git pull and git fetch

Bringing a teammate's work onto your own machine

While Meena was setting up her laptop, Priya pushed three new commits. Meena's local copy has no idea they exist — yet.

THE STORY CONTINUES

Meena cloned the project yesterday. Since then, Priya has pushed new commits to GitHub. Meena's local `main` branch has silently fallen behind, and nothing on her screen will tell her that unless she asks.

git fetch

THE CONCEPT

`git fetch` downloads any new commits, branches, and history from a remote repository — but does **not** touch your working directory or your current branch at all. It simply updates Git's internal record of "what does the remote currently look like?"

WHAT GIT DOES INTERNALLY

After a fetch, the new commits exist locally under a special remote-tracking reference like `origin/main`, completely separate from your own `main`. You can inspect them, compare them, even look at their code — but your own branch and files remain untouched until you explicitly decide to integrate those changes.

SYNTAX

```
git fetch
git fetch origin
```

```
terminal

$ git fetch origin
remote: Enumerating objects: 9, done.
Unpacking objects: 100% (9/9), done.
From https://github.com/college-team/event-management-system
  8b3e1af..d92c1f0  main       -> origin/main
```

WHEN TO USE THIS

When you want to see what has changed remotely *before* deciding whether or how to bring it into your own work — a cautious, look-before-you-leap approach.

COMMON MISTAKES

- Running `git fetch` and expecting your files to have changed — they won't, until you merge or pull.

git pull

THE STORY CONTINUES

Most of the time, though, Meena doesn't want to inspect changes separately – she just wants to be up to date, immediately.

THE CONCEPT

`git pull` is simply `git fetch` immediately followed by `git merge` (Chapter 16) of the fetched changes into your current branch. It downloads new commits *and* integrates them into your working directory in one step.

SYNTAX

```
git pull
git pull origin main
```

```
terminal

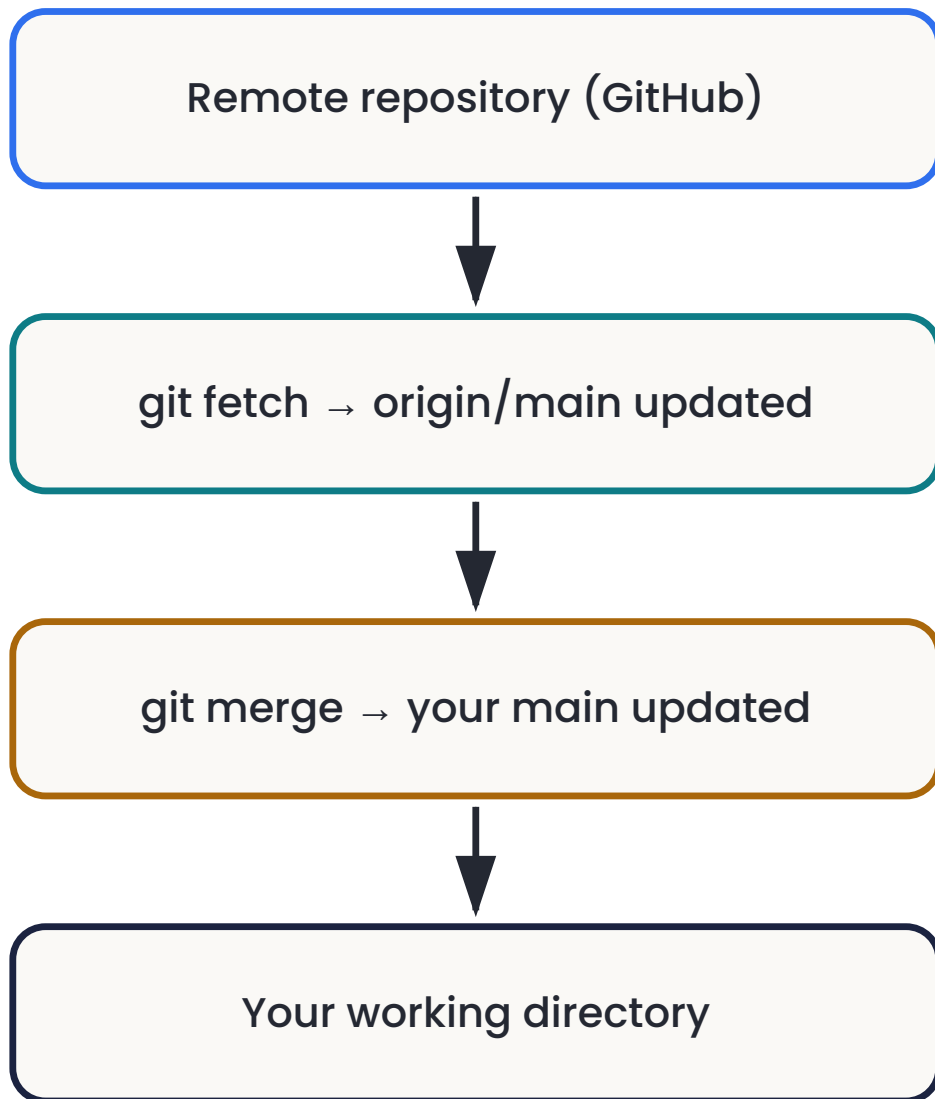
$ git pull
remote: Enumerating objects: 9, done.
Unpacking objects: 100% (9/9), done.
From https://github.com/college-team/event-management-system
   8b3e1af..d92c1f0  main       -> origin/main
Updating 8b3e1af..d92c1f0
Fast-forward
 dashboard.py | 12 ++++++---
 1 file changed, 9 insertions(+), 3 deletions(-)
```

WHEN TO USE THIS

At the start of nearly every work session, and always right before starting new work on a shared branch like `main`.

COMMON MISTAKES

- Forgetting to pull before starting a day's work, then discovering your branch has quietly fallen behind and needs extra work to reconcile.
- Pulling with uncommitted local changes that conflict with incoming ones, which can trigger a merge conflict (Chapter 17) in the middle of what felt like a routine update.



git pull = git fetch + git merge, back to back.

fetch vs. pull: When to Use Each

	git fetch	git pull
Downloads new history?	Yes	Yes
Changes your files/branch?	No	Yes — merges immediately
Best for	Reviewing remote changes safely before integrating them	Quickly catching up when you trust there won't be conflicts

COMMON MISTAKES

- Treating `fetch` and `pull` as interchangeable — `fetch` alone never changes your working files.
- Pulling into a branch with uncommitted changes still sitting around, risking a confusing conflict.

MINI PRACTICE TASK

Practice keeping up to date:

1. In a cloned repository, run `git fetch` and then `git log origin/main --oneline` to see remote commits without touching your own branch.
2. Run `git pull` and confirm your local branch now matches the remote.

PART IV

BRANCHING AND TEAMWORK

Branches

Working in parallel without stepping on each other's code

Arjun needs to build the login screen. Priya needs to build the dashboard. Neither wants to touch `main` until their feature actually works.

THE STORY CONTINUES

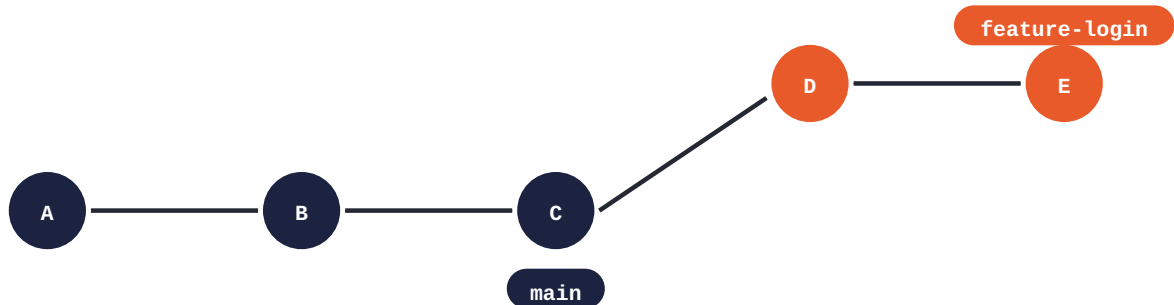
The project is growing, and Arjun and Priya now need to work on two different features at the same time: Arjun on login, Priya on the dashboard. If they both edit the same `main` line of history directly, an unfinished, broken login screen could end up mixed in with Priya's dashboard work, and vice versa.

THE PROBLEM

Without any separation, "work in progress" and "stable, working code" would live in the exact same place. Arjun couldn't safely experiment without risking Priya's stability, and Priya couldn't pull the latest `main` without possibly pulling in Arjun's half-finished login screen too.

THE CONCEPT

A **branch** is an independent, parallel line of development. Creating a branch lets you make commits that don't affect `main` (or anyone else's branch) at all, until you deliberately decide to bring that work back together with a **merge** (Chapter 16).



Arjun's `feature-login` branch splits off from commit C. Commits D and E exist only on his branch until it is merged.

```
git branch <name>
```

THE CONCEPT

Creates a new branch pointer named `<name>`, starting from your current commit — but does **not** switch you onto it. Your current branch remains checked out.

SYNTAX

```
git branch feature-login
```


terminal

```
$ git branch feature-login
$ git branch
* main
  feature-login
```

WHEN TO USE THIS

When you want to create a branch to switch to in a moment, or create several branches up front before starting work on any of them.

COMMON MISTAKES

- Creating a branch and expecting to already be "on" it — `git branch` alone only creates it; you must switch to it separately.

Switching Branches: The Modern Way

```
git switch <branch>
```

THE CONCEPT

`git switch` moves you onto a different existing branch, updating your working directory to match that branch's latest commit.

SYNTAX

```
git switch feature-login
```

terminal

```
$ git switch feature-login
Switched to branch 'feature-login'
```

WHEN TO USE THIS

Whenever you want to move your work to a different, already-existing branch.

```
git switch -c <branch>
```

Creates a new branch *and* switches to it immediately, combining the two previous commands into one — this is the version you will use constantly in practice.

SYNTAX

```
git switch -c feature-login
```

terminal

```
$ git switch -c feature-login
Switched to a new branch 'feature-login'
```

The Older Command: `git checkout`

NOTE

Before `git switch` existed, branch switching (and several other unrelated jobs, like restoring files) all shared one overloaded command: `git checkout`. You will still see it constantly in older tutorials and on Stack Overflow, so it's worth recognising — but `git switch` and `git restore` (Chapter 9) were introduced specifically to split its responsibilities into clearer, less error-prone commands. This book recommends the modern versions.

```
git checkout <branch>
```

The older equivalent of `git switch <branch>`.

SYNTAX

```
git checkout feature-login
```

```
git checkout -b <branch>
```

The older equivalent of `git switch -c <branch>`: create and switch in one step.

SYNTAX

```
git checkout -b feature-login
```

Listing and Deleting Branches

```
git branch -a
```

Lists **all** branches: your local ones, plus every branch that exists on the remote (shown in a different colour, prefixed with `remotes/origin/`).

SYNTAX

```
git branch -a
```

```
terminal
$ git branch -a
* main
  feature-login
  remotes/origin/main
  remotes/origin/feature-dashboard
```

```
git branch -r
```

Lists only the remote-tracking branches, without your local ones.

SYNTAX

```
git branch -r
```

```
git branch -d <branch>
```

THE CONCEPT

Deletes a local branch — but **only** if Git can confirm its work has already been safely merged elsewhere. This is the "safe delete."

SYNTAX

```
git branch -d feature-login
```

terminal

```
$ git branch -d feature-login
Deleted branch feature-login (was a91cd3e).
```

WHEN TO USE THIS

After a branch has been merged (Chapter 16) and its Pull Request approved (Chapter 18), to keep your branch list tidy.

COMMON MISTAKES

- Trying to delete a branch that hasn't been merged yet with `-d`, and being confused by Git's refusal — that refusal is a safety feature, not a bug.

git branch -D <branch>

The **forced** delete. Removes a branch even if its commits have never been merged anywhere — permanently discarding that work.

SYNTAX

```
git branch -D experimental-idea
```

When to use it: Only when you're certain the branch's work is genuinely disposable.

Watch out: Using capital `-D` as a habit instead of lowercase `-d` — the capital version bypasses the exact safety check that protects you from losing unmerged work.

COMMON MISTAKES

- Forgetting which branch you're currently on before making changes — run `git branch` (look for the `*`) or `git status` (top line) to check.
- Committing directly to `main` out of habit, instead of creating a feature branch first.

MINI PRACTICE TASK

Practice branching:

1. From `main`, create and switch to a new branch using `git switch -c`.
2. Make a commit on that branch.
3. Switch back to `main` with `git switch main` and notice your new file/change is not there.
4. Switch back to your branch and confirm it reappears.

Merging

Bringing two lines of history back together

Arjun's login feature works. It's time to bring it back into main, where the rest of the team can build on top of it.

THE STORY CONTINUES

Arjun's `feature-login` branch is finished and tested. Now he wants everything he built there to become part of `main`, so Priya and Rahul can use it too.

git merge <branch>

THE CONCEPT

Run while checked out on the branch you want to update (usually `main`), `git merge <branch>` combines the history of the named branch into your current one.

SYNTAX

```
git switch main
git merge feature-login
```

```
terminal

$ git switch main
$ git merge feature-login
Updating 3f2a91c..a91cd3e
Fast-forward
 login.py | 40 +++++++++++++++++++++++++++++++++++++
 1 file changed, 40 insertions(+)
```

WHEN TO USE THIS

Once a branch's work is complete (and ideally reviewed — see Pull Requests in Chapter 18) and ready to become part of the main line of development.

COMMON MISTAKES

- Running `git merge` while checked out on the wrong branch — always confirm with `git branch` or `git status` which branch you're merging into before running the command.

Fast-Forward Merges

THE CONCEPT

If `main` hasn't moved forward *at all* since the branch was created, Git doesn't need to do any real "combining" — it can simply slide the `main` pointer forward to match the tip of the feature branch. This is called a **fast-forward merge**, and it's exactly what happened in Arjun's example above.

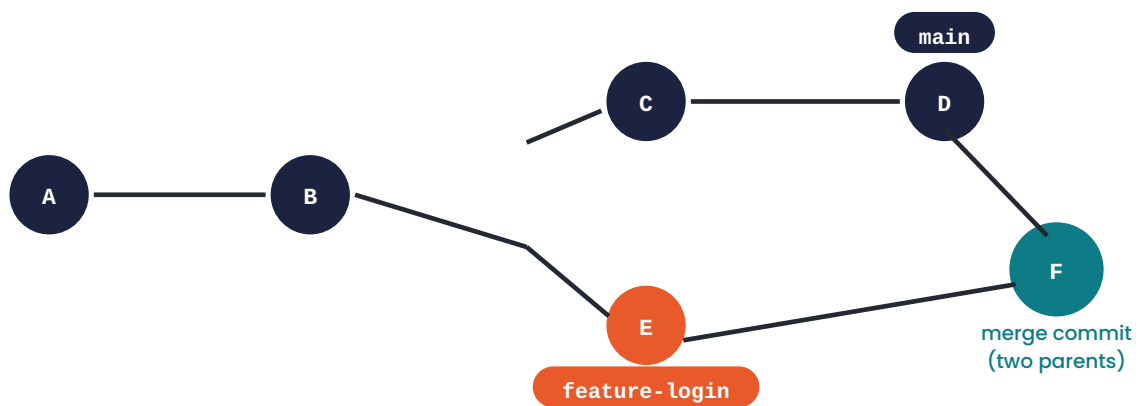


A fast-forward merge: no new commit is created, main's pointer just moves up to D, then E.

Three-Way Merges and Merge Commits

THE CONCEPT

If `main` has moved forward with its own new commits since the branch split off, Git can no longer just slide the pointer — the two histories have genuinely diverged. Git creates a special **merge commit**, which has two parent commits instead of the usual one, weaving both histories back together.



A three-way merge: commit F has two parents — D (the tip of main) and E (the tip of feature-login) — and Git combines both histories into it.

WHAT GIT DOES INTERNALLY

Git calls it a "three-way" merge because it looks at *three* snapshots to combine changes correctly: the common ancestor both branches share (commit B above), the tip of your current branch (D), and the tip of the branch being merged in (E). Comparing all three lets Git figure out exactly what changed on each side since they diverged.

WHEN TO USE THIS

You don't choose between a fast-forward and a three-way merge yourself — Git decides automatically based on whether your current branch has moved since the other branch split off. Either way, the command you type is the same: `git merge <branch> .`

COMMON MISTAKES

- Being surprised by an unexpected merge commit and a text editor opening for its message — that's completely normal any time the branches have diverged; save and close it to continue.
- Assuming every merge will be as clean as a fast-forward — diverged branches that changed the same lines will trigger a merge conflict instead (Chapter 17).

MINI PRACTICE TASK

See both merge types yourself:

1. Create a branch, commit on it, switch back to main (without any new main commits), and merge — confirm it says "Fast-forward."
2. Repeat, but this time make a new commit on main first before merging — confirm a merge commit is created instead, and look at the result with `git log --graph --oneline`.

Merge Conflicts

What to do when Git genuinely can't decide for you

Arjun and Priya have both edited the exact same line. Git refuses to guess who is right — and that's actually a good thing.

THE STORY CONTINUES

Arjun changes a welcome message in `Greeting.java` to say hello to himself while testing. On a different branch, Priya changes the very same line to greet Priya instead. When Arjun tries to merge Priya's branch into his own, Git stops and refuses to guess which greeting is "correct."

THE PROBLEM

Two branches have changed the exact same line of the exact same file in two different, incompatible ways. Git has no way to know which version you actually want — or whether you want some combination of both — so it pauses the merge and asks a human to decide.

Reading a Conflict

```
terminal
$ git merge feature-priya-greeting
Auto-merging Greeting.java
CONFLICT (content): Merge conflict in Greeting.java
Automatic merge failed; fix conflicts and then commit the result.
```

Opening `Greeting.java` now shows special markers inserted directly into the file:

```
<<<<<<< HEAD
    System.out.println("Hello Arjun");
=====
    System.out.println("Hello Priya");
>>>>>> feature-priya-greeting
```

Marker	What it means
<<<<<<< HEAD	Everything below this line, down to the =====, is your current branch's version.
=====	The dividing line between the two conflicting versions.
>>>>>> feature-priya-greeting	Everything above this line (and below =====) is the incoming branch's version.

Resolving the Conflict

THE CONCEPT

Resolving a conflict means manually editing the file to leave it in the state you actually want — which might be Arjun's version, Priya's version, some combination of both, or something else entirely — and then **deleting the conflict markers themselves**. Git will not do this part for you; it is a human decision.

Suppose the team decides on a generic greeting instead. Arjun edits the file to:

```
System.out.println("Hello, and welcome!");
```

with every `<<<<<<`, `=====`, and `>>>>>>` line completely removed.

Completing the Merge

```
terminal

# after manually editing the file to remove the markers
$ git status
You have unmerged paths.
  (fix conflicts and run "git commit")
Unmerged paths:
  both modified:   Greeting.java

$ git add Greeting.java
$ git commit
[main 7d4c2ab] Merge branch 'feature-priya-greeting'
```

THE CONCEPT

Notice that resolving a conflict uses commands you already know: `git status` to see what needs fixing, `git add` to mark the file as resolved once you've edited it, and `git commit` to finalise the merge. Nothing new to memorise — conflicts are resolved with the same tools as everything else.

Backing Out of a Merge Entirely

```
git merge --abort
```

THE CONCEPT

If a conflict looks too messy, or you realise you merged the wrong branch entirely, this command cancels the in-progress merge completely and returns every file to exactly how it looked right before you ran `git merge`.

SYNTAX

```
git merge --abort
```

```
terminal

$ git merge --abort
$ git status
On branch main
nothing to commit, working tree clean
```

WHEN TO USE THIS

The moment you decide a conflict resolution isn't going well, before you've run `git commit` to finish the merge.

COMMON MISTAKES

- Trying to use `--abort` after already committing the merge — by that point the merge is finished; use the undo techniques from Chapter 9 instead.

COMMON MISTAKES

- Leaving conflict markers in a file by accident and committing them as if they were real code — always search the file for `<<<<<<` before staging.
- Panicking and deleting the whole file instead of resolving the conflict inside it.
- Forgetting to `git add` the resolved file before running `git commit`.

MINI PRACTICE TASK

Create and resolve a conflict on purpose:

1. Create two branches from the same commit, and edit the exact same line of the exact same file differently on each.
2. Merge one branch into the other and confirm you get a CONFLICT message.
3. Open the file, resolve it by hand, remove the markers, then `git add` and `git commit` to finish.

GitHub Pull Requests

Asking for a second pair of eyes before code joins main

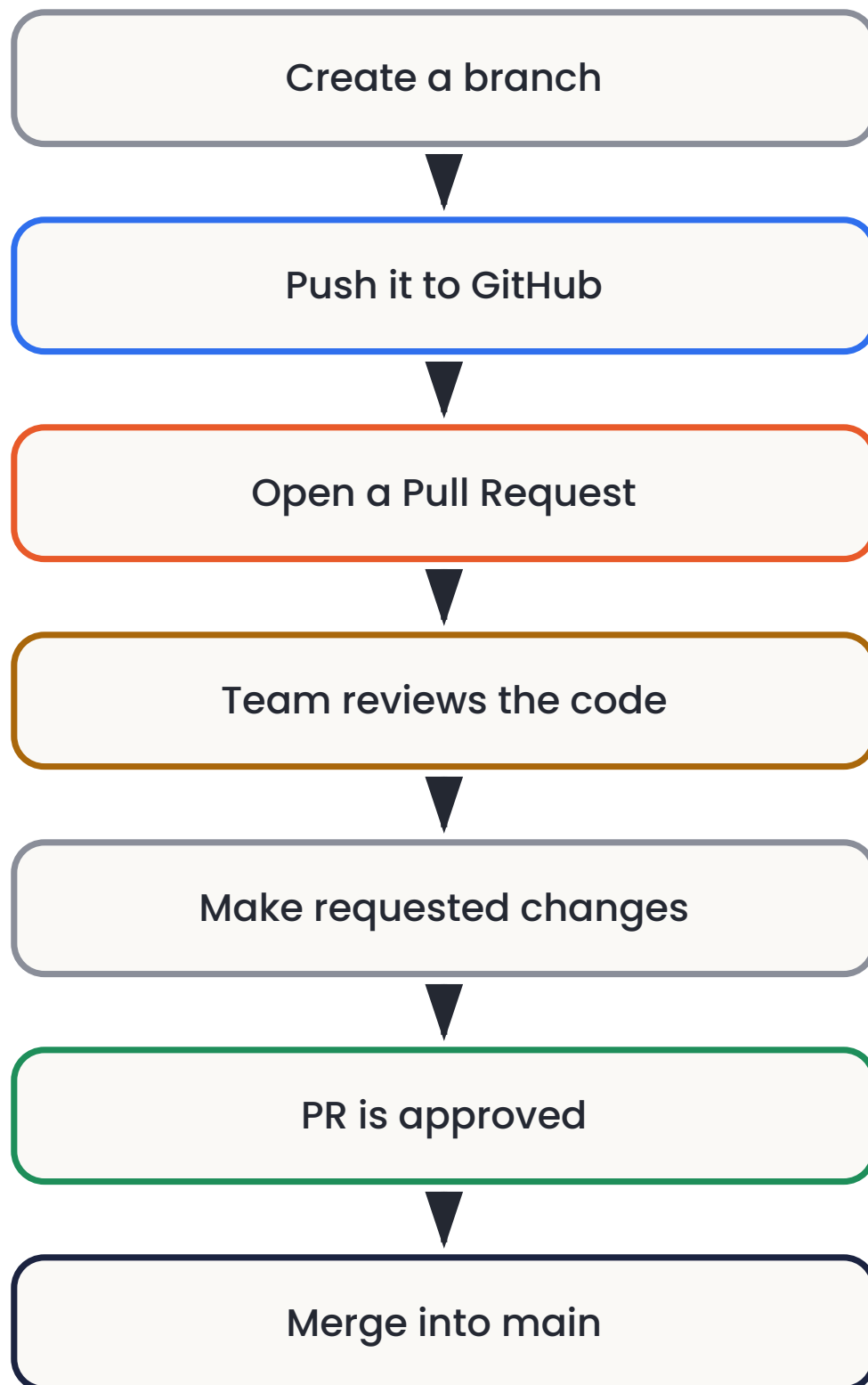
Rahul's backend feature works on his machine. Before it becomes everyone's problem, the team wants a chance to look at it first.

THE STORY CONTINUES

Rahul finishes the attendance-export feature on his own branch. Instead of merging it straight into `main` himself, he opens a **Pull Request** on GitHub — a formal request asking the rest of the team to review his code before it becomes part of the shared project.

THE CONCEPT

A **Pull Request** (PR) is GitHub's tool for proposing that changes from one branch be merged into another, along with a space for discussion, comments, and review — all before the merge actually happens.



The Pull Request lifecycle, from a fresh branch to a merged feature.

Opening a Pull Request

```
terminal

$ git switch -c feature-export
# ...work happens, commits are made...
$ git push origin feature-export
remote: Create a pull request for 'feature-export' on GitHub by visiting:
remote:  https://github.com/college-team/event-management-system/pull/new/feature-export
```

Following that link on GitHub opens a form: a title, a description of what changed and why, and a button labelled **Create pull request**.

Pull Request Vocabulary

Term	Meaning
Reviewer	A teammate asked to look over the proposed changes before they're merged.
Comment	Feedback left on the PR overall, or on one specific line of code.
Requested changes	A reviewer's verdict meaning "don't merge yet — please fix these things first."
Approval	A reviewer's verdict meaning "this looks good to merge as-is."
Merge	The final step: the PR's branch is combined into the target branch (usually <code>main</code>).

THE STORY CONTINUES

Meena reviews Rahul's PR and leaves a comment on one specific line: "This loop can be simplified with a list comprehension." Rahul updates his branch and pushes again — the PR updates automatically, no need to open a new one. Once Meena is satisfied, she approves it, and Rahul (or Meena) clicks **Merge pull request**.

Merge Strategies

Strategy	What it does	When it's useful
Merge commit	Creates a standard merge commit with two parents, preserving the branch's individual commits exactly as they were made.	When you want a complete, detailed history, including every intermediate commit.
Squash and merge	Combines every commit on the branch into a single new commit on the target branch.	When a branch has many small, messy "WIP" commits that don't need to be preserved individually.
Rebase and merge	Replays the branch's commits one by one directly onto the tip of the target branch, with no merge commit at all.	When you want a clean, perfectly linear history with no merge commits (see Chapter 24 for how rebase works).

COMMON MISTAKES

- Merging your own PR without waiting for a review, defeating the entire purpose of opening one.
- Writing a PR description so vague that reviewers have to guess what changed and why.
- Force-pushing over review comments in a way that makes a reviewer's earlier feedback hard to trace back to the code it was about.

MINI PRACTICE TASK

Open your first real Pull Request:

1. Push a feature branch to a repository you control.
2. Open a Pull Request on GitHub comparing it to main, with a clear title and description.
3. If you're working with a partner, ask them to leave at least one comment, then respond to it with a new commit before merging.

Forks

Contributing to a project you don't have permission to edit directly

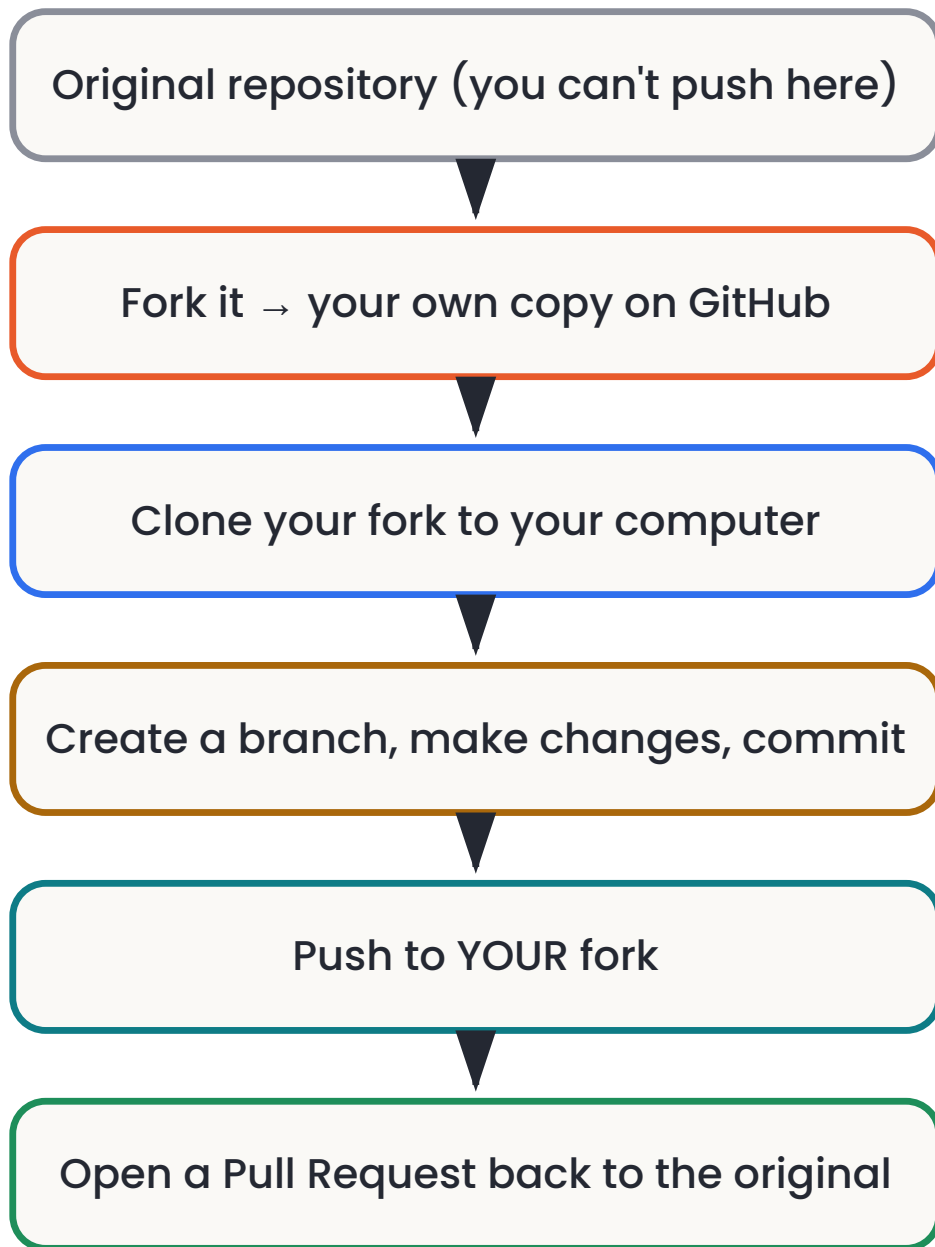
Arjun wants to fix a typo in a popular open-source library's documentation — a project he has never been given write access to.

THE STORY CONTINUES

Browsing GitHub, Arjun notices a small documentation typo in a well-known open-source project. He doesn't have permission to push directly to it — nobody random on the internet does. Kabir explains: "That's exactly what forking is for."

THE CONCEPT

A **fork** is your own personal, complete copy of someone else's repository, created on GitHub under your own account. You have full write access to your fork, even though you have none on the original.



The open-source contribution path: fork, clone, branch, push, then propose your change back via a Pull Request.

fork vs. clone: Two Different Jobs

	Fork	Clone
Where it happens	On GitHub's servers (a website action)	On your own computer (a terminal action)
What it creates	A brand-new repository under your GitHub account	A local copy of a repository (any repository – yours, a teammate's, or a fork)
Typical next step	Clone your new fork to your own computer to start editing	Start committing and, if you have write access, pushing directly

TIP

In practice, contributing to an open-source project almost always uses *both*: first fork on GitHub's website, then clone your fork's URL to your own computer to actually write code.

Why Open-Source Projects Rely on Forks

THE CONCEPT

Popular open-source projects can have thousands of occasional contributors. Giving every one of them direct push access to the original repository would be an enormous, unmanageable security risk. Forking solves this elegantly: anyone can propose a change from their own isolated copy, and the project's maintainers stay in full control of what actually gets merged, via review on the resulting Pull Request.

COMMON MISTAKES

- Trying to push directly to the original repository instead of your fork, and being confused by a permissions error.
- Forgetting to sync your fork with the original repository over time, causing your fork to fall further and further behind (Chapter 25 covers managing an `upstream` remote for exactly this).

MINI PRACTICE TASK

Make your first open-source-style contribution:

1. Find a small, friendly open-source repository (many have a "good first issue" label).
2. Fork it to your own GitHub account.
3. Clone your fork, create a branch, make a small improvement, and push it to your fork.
4. Open a Pull Request from your fork back to the original repository.

PART V

GITHUB PROJECT FEATURES

GitHub Issues

Tracking bugs, ideas, and tasks without a single sticky note

The team keeps forgetting which bugs still need fixing. GitHub Issues turns that mental list into a shared, trackable one.

THE STORY CONTINUES

The Event Management System has grown enough that the team can no longer keep track of bugs and to-dos in their heads, or in scattered WhatsApp messages. Priya sets up **GitHub Issues** for the repository so every bug, idea, and task lives in one visible, shared place.

THE CONCEPT

An **Issue** is a single trackable item on GitHub — a bug report, a feature request, or a task — with a title, a description, a status (open or closed), and space for discussion in comments.

Anatomy of a Good Issue

Field	Purpose	Example
Title	A short, searchable summary	"Attendance percentage shows negative numbers for absent-only students"
Description	Steps to reproduce a bug, or details of a requested feature	"Steps: 1) Mark a student absent for all classes. 2) Open their report. Expected: 0%. Actual: -100%."
Labels	Categorise and filter issues at a glance	<code>bug</code> , <code>enhancement</code> , <code>good first issue</code>
Assignee	Who is responsible for resolving it	Rahul
Milestone	Which release or sprint this issue belongs to	"v1.0 Launch"

Common Labels and What They Signal

Label	Meaning
<code>bug</code>	Something is broken and needs fixing.
<code>enhancement</code>	A new feature or improvement, not a fix.
<code>documentation</code>	The README or other docs need updating.
<code>good first issue</code>	Small and well-scoped — ideal for a newcomer to the project, like Meena in her first week.
<code>help wanted</code>	The team is actively looking for someone to pick this up.

THE STORY CONTINUES

Priya opens an Issue titled "Attendance percentage shows negative numbers," labels it `bug`, and assigns it to Rahul. Rahul fixes it on a branch, and in his commit message writes `Fixes #14` — GitHub automatically links the commit to Issue #14 and closes it the moment that commit reaches `main`.

TIP

Referencing an issue number with `#14` anywhere in a commit message or Pull Request description automatically creates a clickable link on GitHub. Prefixing it with a keyword like `Fixes`, `Closes`, or `Resolves` goes one step further: merging that PR automatically closes the issue too.

Milestones

THE CONCEPT

A **milestone** groups several related Issues and Pull Requests toward a shared goal or deadline, such as a release — with GitHub automatically showing how many of its issues are still open versus closed, as a simple progress bar.

COMMON MISTAKES

- Writing a bug report with no steps to reproduce it, leaving the assignee to guess what actually went wrong.
- Never closing resolved issues, letting the Issues tab fill up with stale, already-fixed problems.
- Using Issues for real-time chat instead of quick discussion — that's what a team chat tool is for; Issues should stay focused on trackable, actionable items.

MINI PRACTICE TASK

Set up Issues for a project of your own:

1. Open an Issue describing a real bug or improvement idea for one of your repositories.
2. Add a label and, if you're on a team, an assignee.
3. Fix it on a branch, and reference the issue number (like `Fixes #1`) in your commit message or PR description.
4. Merge the fix and confirm the Issue closed automatically.

GitHub README

Writing the first (and sometimes only) thing anyone reads

A visitor lands on the team's repository for the very first time. The README is the only thing standing between them and total confusion.

THE STORY CONTINUES

A junior student from another class wants to try running the Event Management System project. She opens the GitHub page and finds... nothing but code files and no explanation. She leaves within thirty seconds. "A great project with no README," Kabir says, "is invisible to almost everyone who might have used it."

THE CONCEPT

GitHub automatically renders `README.md` directly on a repository's homepage. It is, by far, the most-read file in most projects — often the *only* file a visitor reads before deciding whether to explore further.

A Professional README Structure

Section	What goes here
# Project Name	The project's name as a top-level heading.
## Description	One or two sentences: what does this project do, and for whom?
## Features	A bullet list of the main things the project can do.
## Technologies Used	Languages, frameworks, and major libraries.
## Installation	Exact, copy-pasteable commands to get it running locally.
## Usage	How to actually use it once installed — example commands or screenshots of the flow.
## Screenshots	Images showing the project in action (optional, but very effective).
## Contributors	Who built it.
## License	What others are legally allowed to do with your code.

Example: A Student Project README

```
# Student Attendance System

## Description
A simple web app for tracking daily student attendance and generating
percentage reports for a college classroom.

## Features
- Mark daily attendance per student
- Automatic percentage calculation
- Exportable attendance reports

## Technologies Used
Python, Flask, SQLite

## Installation
```bash
git clone https://github.com/arjunmenon/student-attendance-system.git
cd student-attendance-system
pip install -r requirements.txt
```

## Usage
Run `python app.py` and open http://localhost:5000

## Contributors
Arjun Menon

## License
MIT
```

TIP

The `.md` extension stands for **Markdown**. Lines starting with `#` become headings, lines starting with `-` become bullet points, and text wrapped in triple backticks becomes a formatted code block — all without needing any special editor.

COMMON MISTAKES

- Leaving the default, empty README GitHub generates, with no real project description at all.
- Writing installation instructions that skip a step you personally have memorised, leaving a new contributor stuck.
- Letting the README go stale as the project changes, so it no longer matches how the project actually works.

MINI PRACTICE TASK**Write a real README:**

1. Pick one of your existing repositories.
2. Write a complete README following the structure above.
3. Ask a classmate to follow only your Installation section on a fresh machine, and fix anything that confused them.

Git Tags and Releases

Marking the exact moment a version became official

Version 1.0 of the Event Management System is finally ready to demo to the faculty. The team wants a permanent marker for exactly this point in history.

THE STORY CONTINUES

After weeks of work, the team's project is stable enough to demo. Priya wants to be able to return to this exact commit forever, even after dozens more commits pile on top of it later — and to give it a proper, memorable name: `v1.0.0`.

THE CONCEPT

A **tag** is a permanent, named pointer to one specific commit — unlike a branch, a tag never moves forward as new commits are added. It marks a point in history, forever.

```
git tag <name>
```

THE CONCEPT

Creates a lightweight tag — essentially just a fixed label pointing at your current commit.

SYNTAX

```
git tag v1.0.0
```

 terminal

```
$ git tag v1.0.0
$ git tag
v1.0.0
```

WHEN TO USE THIS

For a quick, informal marker without needing extra details attached.

```
git tag -a <name> -m "message"
```

THE CONCEPT

Creates an **annotated** tag — a fuller object storing the tagger's name, date, and a message, similar to a commit. This is the recommended kind of tag for anything meaningful, like a release.

SYNTAX

```
git tag -a v1.0.0 -m "First stable release"
```

```
terminal

$ git tag -a v1.0.0 -m "First stable release"
$ git show v1.0.0
tag v1.0.0
Tagger: Priya Sharma <priya@example.com>
Date: Tue Sep 15 18:20:00 2026 +0530

    First stable release
```

WHEN TO USE THIS

Whenever you're marking an official release, exactly like this chapter's example.

COMMON MISTAKES

- Forgetting that tags, unlike branches, don't move — if you need to "redo" a tagged release, you create a new tag rather than moving the old one.

Pushing Tags to GitHub

Tags are **not** included automatically by a plain `git push` — they need to be pushed explicitly.

```
git push origin v1.0.0
```

Pushes one specific tag to the remote.

SYNTAX

```
git push origin v1.0.0
```

```
git push origin --tags
```

Pushes every tag that exists locally but not yet on the remote, in one command.

SYNTAX

```
git push origin --tags
```

Watch out: Assuming a normal `git push` also uploads your tags — it does not; tags always need one of the two commands above.

Semantic Versioning

THE CONCEPT

Most projects tag releases using **semantic versioning**: `MAJOR.MINOR.PATCH`, for example `v1.0.0`.

| Position | Increment when... |
|----------------------|--|
| MAJOR (1.x.x) | You make incompatible, breaking changes. |
| MINOR (x.1.x) | You add new functionality that doesn't break existing usage. |
| PATCH (x.x.1) | You make backward-compatible bug fixes only. |

GitHub Releases

THE CONCEPT

A **GitHub Release** takes a tag one step further: on GitHub's website, it turns a tag into a shareable page with formatted release notes, and optionally attaches downloadable files (like a compiled app or a zip of the source). Every Release is backed by a tag, but not every tag has a Release built on top of it.

TIP

To create one: on GitHub, go to your repository's **Releases** tab, click **Draft a new release**, choose (or create) a tag like `v1.0.0`, write release notes describing what changed, and publish.

COMMON MISTAKES

- Using plain (lightweight) tags for official releases instead of annotated ones, losing the tagger and date information.
- Forgetting to push tags, so a release exists locally but nobody else can see it on GitHub.
- Inconsistent version numbers that don't follow semantic versioning, confusing anyone trying to judge whether an update is safe to install.

MINI PRACTICE TASK

Tag your first release:

1. In a project with at least a few commits, create an annotated tag named `v1.0.0`.
2. Push it to GitHub with `git push origin v1.0.0`.
3. On GitHub, create a Release based on that tag with a short description of what it includes.

PART VI

ADVANCED LOCAL GIT

Stashing

Setting messy, half-finished work aside without committing it

Arjun is halfway through a risky experiment when an urgent bug report comes in on a completely different branch. He isn't ready to commit — or lose — his half-finished work.

THE STORY CONTINUES

Arjun is midway through rewriting the login form when Kabir messages the group: a critical bug is breaking the dashboard for everyone, and it needs a fix right now. Arjun's login changes are messy and don't even compile yet — he doesn't want to commit them, but switching branches with uncommitted changes sitting around can cause problems.

THE PROBLEM

Git normally won't let you switch branches if doing so would overwrite uncommitted changes that conflict with the target branch — and even when it does allow it, carrying half-finished, broken code onto a different branch is messy and risky.

git stash

THE CONCEPT

`git stash` takes all of your uncommitted changes — staged and unstaged — and tucks them away in a hidden, temporary storage area, leaving your working directory clean, exactly as it was at your last commit. You can bring those changes back at any time, even on a different branch.

SYNTAX

`git stash`

terminal

```
$ git stash
Saved working directory and index state WIP on feature-login: a91cd3e Add login form
$ git status
nothing to commit, working tree clean
```

WHEN TO USE THIS

Any time you need to switch context — change branches, pull updates, or help with something urgent — without committing unfinished, messy work.

COMMON MISTAKES

- Forgetting you have something stashed and being confused later why old changes seem to have vanished — `git stash list` always shows what's waiting.

git stash push -m "WIP login"

The fuller form of `git stash`, letting you attach a descriptive message — essential once you have more than one stash saved at a time.

SYNTAX

```
git stash push -m "WIP login"
```

terminal

```
$ git stash push -m "WIP login form redesign"
```

git stash list

Shows every stash currently saved, most recent first, along with the branch and message it was saved from.

SYNTAX

```
git stash list
```

terminal

```
$ git stash list
stash@{0}: On feature-login: WIP login form redesign
stash@{1}: On feature-dashboard: WIP chart colours
```

git stash pop

THE CONCEPT

Restores the most recent stash back into your working directory **and removes it from the stash list**, since you're presumably done needing it saved separately.

SYNTAX

```
git stash pop
```

terminal

```
$ git stash pop
On branch feature-login
Changes not staged for commit:
  modified:   login.py
Dropped stash@{0}
```

WHEN TO USE THIS

When you're back on the right branch and ready to continue exactly where you left off.

COMMON MISTAKES

- Popping a stash onto a branch whose files have changed enough to conflict with the stashed changes — this can trigger a conflict just like a merge conflict, resolved the same way (Chapter 17).

git stash apply

Like `git stash pop`, but **keeps** the stash in the list afterward instead of removing it — useful if you want to apply the same stashed changes to more than one branch.

SYNTAX

```
git stash apply
```

git stash drop

Permanently deletes one specific stash without applying it.

SYNTAX

```
git stash drop stash@{0}
```

git stash clear

Permanently deletes *every* stash at once.

SYNTAX

```
git stash clear
```

Watch out: Running `git stash clear` without checking `git stash list` first — there's no confirmation prompt, and cleared stashes cannot be recovered through normal Git commands.

COMMON MISTAKES

- Forgetting stashed work exists entirely, sometimes for weeks.
- Using stash as a long-term storage habit instead of just committing to a branch — stashes are meant to be short-lived.

MINI PRACTICE TASK

Practice the stash-and-switch workflow:

1. Make some uncommitted edits to a tracked file.
2. Run `git stash push -m "testing"` and confirm `git status` is clean.
3. Switch to a different branch and back.
4. Run `git stash pop` and confirm your edits return.

Git Rebase

Rewriting history to look like it happened in a straight line

Kabir shows Arjun a second way to bring a feature branch up to date — one that produces a perfectly straight, easy-to-read history, with one very important warning attached.

THE STORY CONTINUES

Arjun's `feature-login` branch has fallen behind `main`, which has moved forward with several new commits. He could `merge main` into his branch (Chapter 16), which would create a merge commit — or he could **rebase** instead, for a cleaner-looking history.

```
git rebase main
```

THE CONCEPT

Rebasing takes every commit unique to your current branch and **replays** them, one at a time, on top of the latest commit on `main` — as if you had started your branch from today's `main` all along, instead of from wherever it actually branched off.

WHAT GIT DOES INTERNALLY

Rebasing doesn't literally "move" your existing commits — it creates *brand-new* commits with the same changes and messages, but new parent links and, critically, new commit hashes. The old commits still technically exist for a short while but are abandoned; your branch pointer now refers to the new ones.

SYNTAX

```
git switch feature-login
git rebase main
```

terminal

```
$ git switch feature-login
$ git rebase main
Successfully rebased and updated refs/heads/feature-login.
```

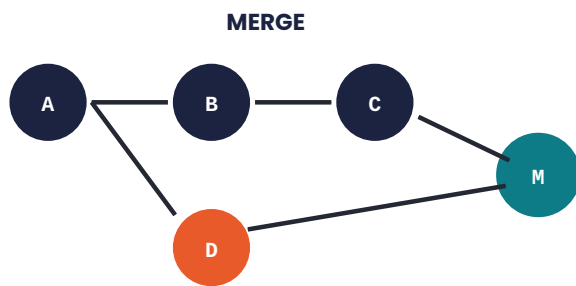
WHEN TO USE THIS

When you want a clean, linear project history with no merge commits — commonly used to update a personal feature branch before opening a Pull Request.

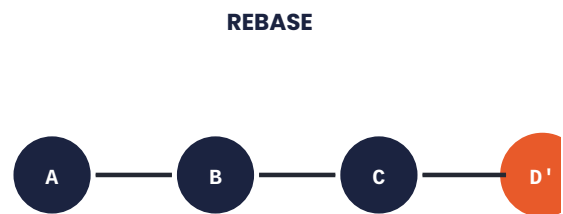
COMMON MISTAKES

- Rebasing a branch that teammates have already pulled and built work on top of — see the warning below.

Rebase vs. Merge



History branches, then a merge commit joins it back



D is replayed as a new commit D' directly after C — one straight

Merge preserves exactly what happened, including the branch shape. Rebase rewrites history into a straight line, as if D had been written after C all along.

DANGER ZONE

Never rebase commits that other people already have. Because rebasing creates brand-new commits with new hashes, anyone who already pulled the *old* versions of those commits now has a history that has permanently diverged from yours — fixing it requires confusing manual intervention for everyone involved. A simple rule that keeps you safe: only rebase branches that are **entirely your own and not yet pushed**, or that you are certain nobody else has based any work on.

Interactive Rebase

```
git rebase -i HEAD~3
```

Opens an interactive editor listing your last three commits, letting you reorder them, combine ("squash") several messy commits into one, reword commit messages, or drop commits entirely — before that branch is ever shared with anyone.

SYNTAX

```
git rebase -i HEAD~3
```

```

terminal

$ git rebase -i HEAD~3
pick a91cd3e Add login form
pick 6f4d2ab WIP
pick 9c1e0da Fix typo in WIP commit
# change 'pick' to 'squash' (or 's') to combine a commit into the one above it
  
```

When to use it: Before opening a Pull Request, to clean up a messy string of "WIP" commits into one or two clear, well-described ones.

COMMON MISTAKES

- Rebasing a branch that has already been pushed and pulled by teammates.
- Using `rebase` as a reflex without understanding it rewrites commit hashes, then being confused when a subsequent push is rejected (Chapter 30 covers the fix, `git push --force-with-lease`, and why it must be used carefully).
- Confusing rebase's "replay" with merge's "combine" — they solve the same practical problem but produce very different-looking histories.

MINI PRACTICE TASK

Practice rebasing safely:

1. On a personal, unpushed branch, make two or three commits, then use `git rebase -i HEAD~3` to squash them into one.
2. Create a feature branch, let 'main' move forward with a new commit, then rebase your branch onto main and compare the resulting history (`git log --graph --oneline`) to what a regular merge would have produced.

Remote Management

Working with more than one remote repository at once

Arjun's fork of an open-source project (Chapter 19) needs to stay in sync with the original — which means talking to two remotes, not just one.

THE STORY CONTINUES

Arjun forked an open-source project weeks ago and it has moved on significantly since then. His fork only knows about `origin` (his own copy) — it has no idea the original project even exists as a separate remote yet.

git remote

THE CONCEPT

Lists the short names of every remote configured for the current repository.

SYNTAX

```
git remote
```

terminal

```
$ git remote
origin
```

git remote -v

The verbose version: lists every remote's name *and* its full URL, for both fetching and pushing.

SYNTAX

```
git remote -v
```

terminal

```
$ git remote -v
origin https://github.com/arjunmenon/cool-library.git (fetch)
origin https://github.com/arjunmenon/cool-library.git (push)
```

git remote add <name> <url>

Registers a new remote under a given name (already covered in Chapter 11 for `origin`).

SYNTAX

```
git remote add upstream https://github.com/original-author/cool-library.git
```

git remote remove <name>

Removes a remote's configuration entirely. This does not delete anything on the actual server — it only forgets the

connection locally.

SYNTAX

```
git remote remove old-origin
```

```
git remote rename <old> <new>
```

Renames a remote locally.

SYNTAX

```
git remote rename origin old-origin
```

```
git remote set-url <name> <url>
```

Changes the URL a remote points to — useful if a repository moves, or if you switch from HTTPS to SSH (Chapter 28) without wanting to re-clone everything.

SYNTAX

```
git remote set-url origin git@github.com:arjunmenon/cool-library.git
```

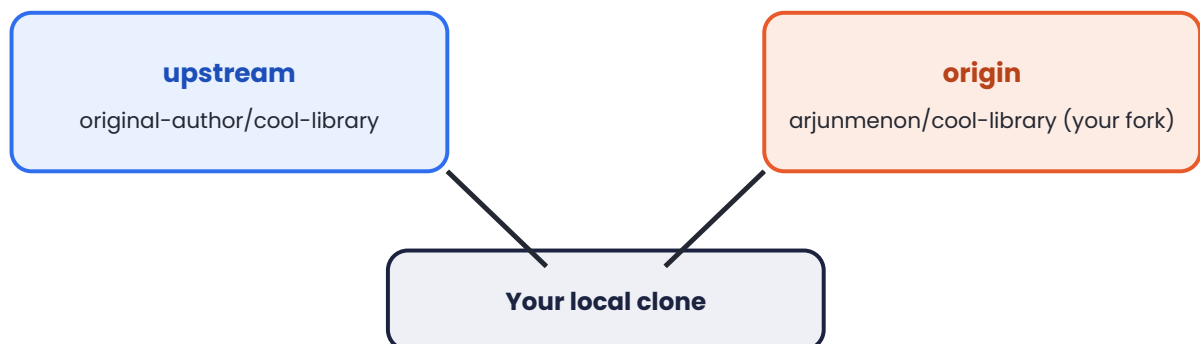
origin and upstream: The Open-Source Convention

THE CONCEPT

When working from a fork, the convention is to name your own fork's remote **origin** (set automatically when you clone your fork) and add the original project as a second remote named **upstream**.

terminal

```
$ git remote add upstream https://github.com/original-author/cool-library.git
$ git fetch upstream
$ git switch main
$ git merge upstream/main
# your fork's main branch is now caught up with the original project
$ git push origin main
# and now your fork on GitHub reflects that too
```



Your local clone can talk to two remotes at once: **upstream** (the original project, fetch-only for most contributors) and **origin** (your own fork, where you push).

COMMON MISTAKES

- Trying to push directly to `upstream` without write access, instead of pushing to `origin` and opening a Pull Request.
- Forgetting to periodically fetch and merge from `upstream`, letting a fork drift further and further out of date.

MINI PRACTICE TASK

Manage remotes on a forked project:

1. Fork any public repository and clone your fork.
2. Add the original repository as a remote named `upstream`.
3. Run `git remote -v` and confirm you now see both `origin` and `upstream`.
4. Fetch from `upstream` and merge its main branch into your own.

PART VII

REAL-WORLD WORKFLOWS

GitHub Collaboration Workflow

Putting every piece together on one real team project

Arjun, Priya, Rahul, and Meena are now building the College Event Management System together. Here is exactly what a real week on this team looks like, command by command.

THE STORY CONTINUES

The team has taken on their biggest project yet: a **College Event Management System** where students can browse upcoming events and register for them, and club coordinators can post new ones. Meena has just joined, and it's time to see the entire Git and GitHub workflow this book has taught, used together, exactly as a real team would use it.

Step 1: Clone the Repository

```
Meena's terminal

$ git clone https://github.com/college-team/event-management-system.git
$ cd event-management-system
```

Step 2: Create a Feature Branch

Meena has been asked to build the event registration confirmation email. She starts from an up-to-date `main`.

```
terminal

$ git switch main
$ git pull
$ git switch -c feature-confirmation-email
```

Step 3–5: Make Changes, Check Status, Stage

```
terminal

# ...Meena writes the email-sending code...
$ git status
Untracked files:
  email_service.py
Changes not staged for commit:
  modified:   registration.py
$ git add .
$ git status
Changes to be committed:
  new file:   email_service.py
  modified:   registration.py
```

Step 6–7: Commit and Push the Branch

terminal

```
$ git commit -m "Send confirmation email on event registration"
$ git push origin feature-confirmation-email
remote: Create a pull request for 'feature-confirmation-email' on GitHub by visiting:
remote:   https://github.com/college-team/event-management-system/pull/new/feature-confirmation-email
```

Step 8–9: Open a Pull Request and Get Reviewed

THE STORY CONTINUES

Meena opens a Pull Request titled "Send confirmation email on event registration." She requests a review from Rahul, since he wrote the original registration logic. Rahul reviews it on GitHub and leaves a comment: "Looks good, but let's also handle the case where the student's email is missing." — and marks the PR **Requested changes** instead of approving it yet.

Step 10: Make the Requested Changes

terminal

```
# Meena is still on her feature-confirmation-email branch
$ git add email_service.py
$ git commit -m "Handle missing email address gracefully"
$ git push
# the existing Pull Request updates automatically — no need to open a new one
```

Rahul reviews again, is satisfied, and clicks **Approve**.

Step 11: Merge the Pull Request

On GitHub, Meena (or Rahul) clicks **Merge pull request**. The `feature-confirmation-email` branch's commits become part of `main`.

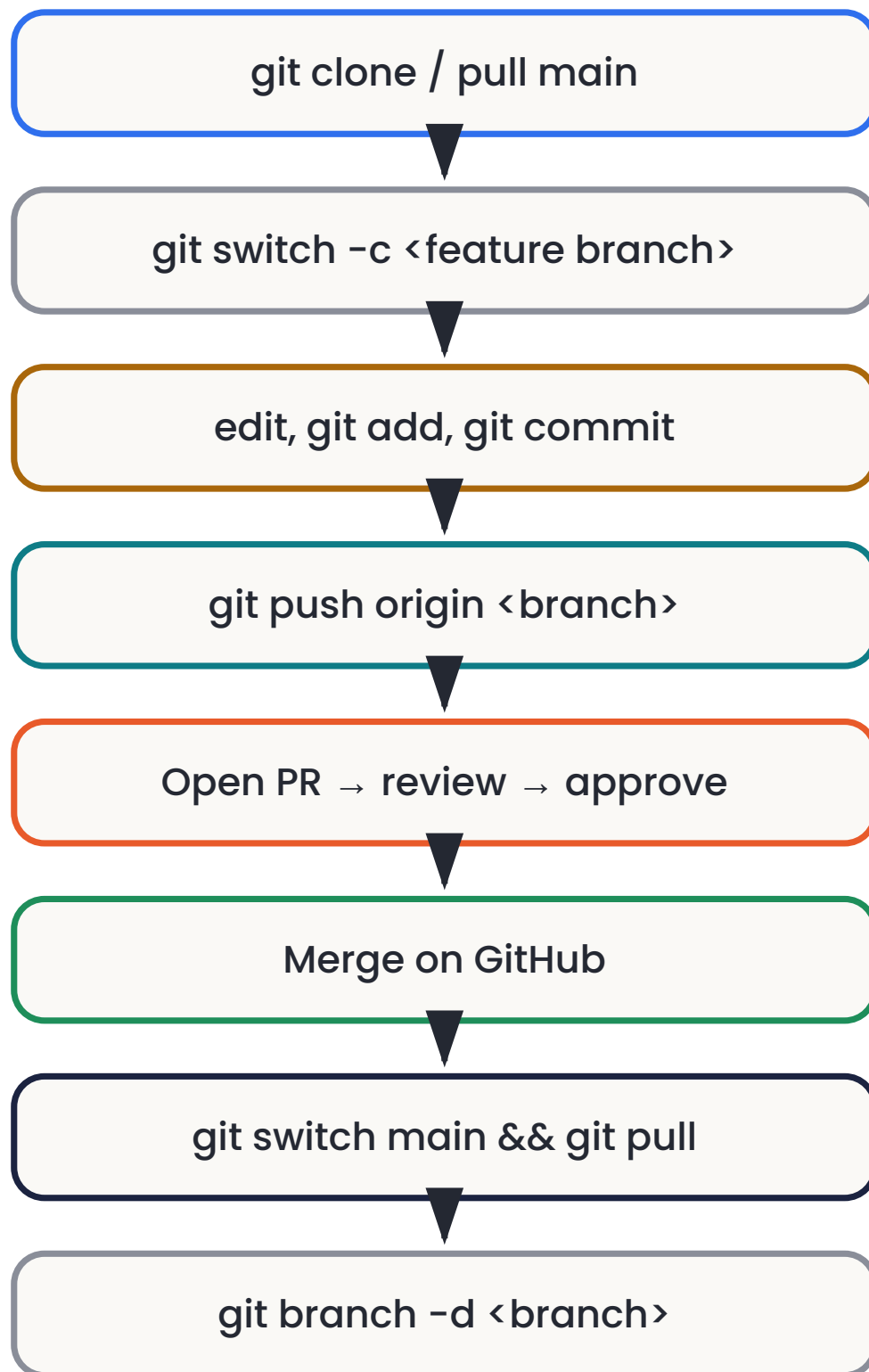
Step 12–13: Update Local Main and Delete the Old Branch

terminal

```
$ git switch main
$ git pull
Fast-forward
$ git branch -d feature-confirmation-email
Deleted branch feature-confirmation-email (was 9d2f4ab).
$ git push origin --delete feature-confirmation-email
# tidies up the now-merged branch on GitHub too
```

THE CONCEPT

Notice that every single step in this entire workflow — the whole reason this book exists — was a command you already know: `clone`, `switch`, `pull`, `status`, `add`, `commit`, `push`, and `branch -d`, plus opening and merging a Pull Request on GitHub itself. There is no secret thirty-seventh command professional teams use that this book hasn't already covered.



The complete team collaboration loop – repeated for every single feature, by every team member, for the life of the project.

COMMON MISTAKES

- Committing directly to `main` instead of working on a feature branch.
- Forgetting to pull `main` before branching off it, starting a feature branch that's already out of date.
- Leaving merged branches lying around indefinitely instead of deleting them once their PR is merged.

MINI PRACTICE TASK

Run this entire loop yourself, ideally with a partner:

1. Clone a shared repository (or simulate one with two folders on your own machine).
2. Create a feature branch, make a change, and push it.
3. Open a real Pull Request and have your partner leave at least one review comment.
4. Address the comment, get it approved, merge it, then clean up your local and remote branches.

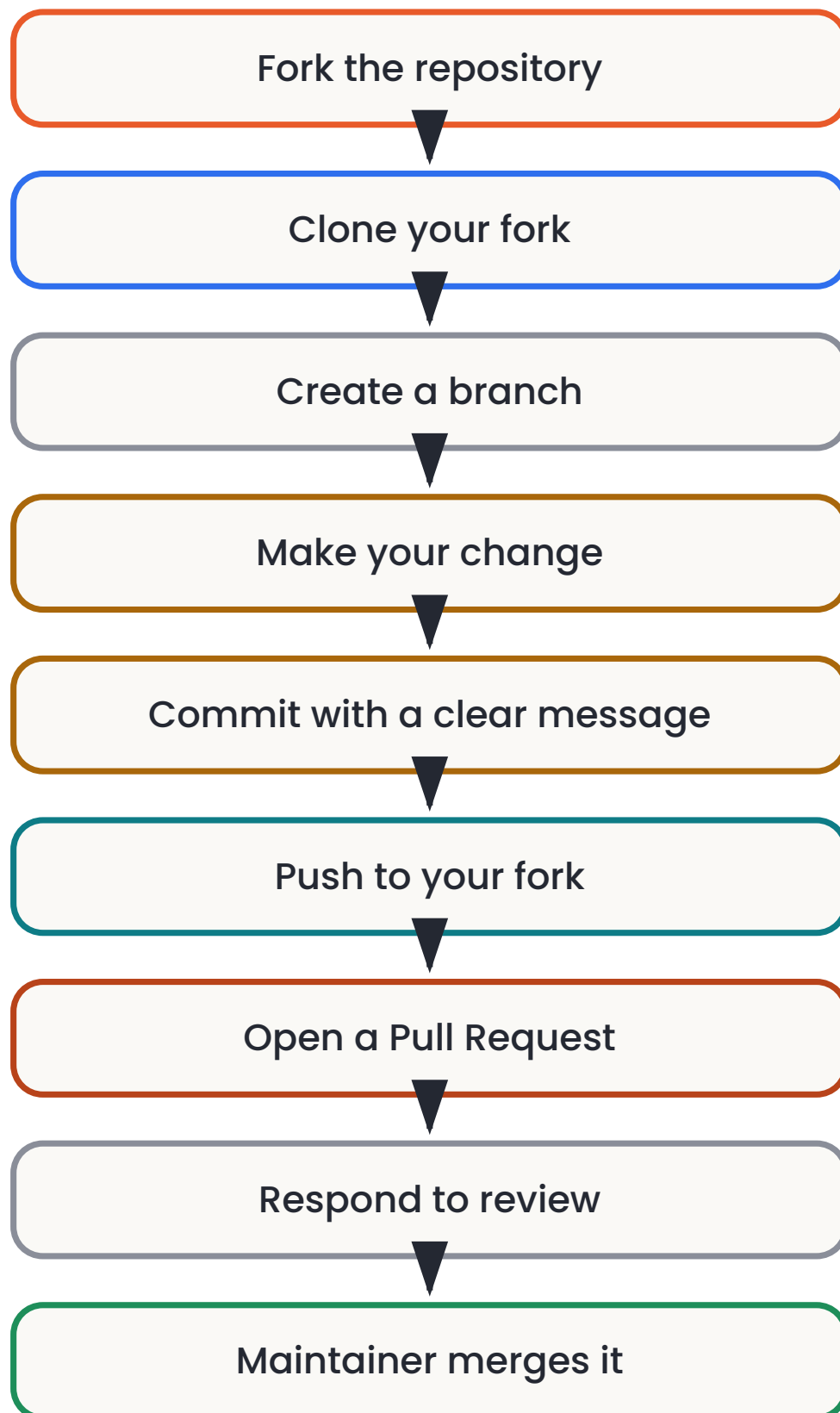
GitHub Open Source Workflow

Contributing to a project the world can see

Arjun wants to fix that documentation typo from Chapter 19 for real. This time, he follows the etiquette that makes maintainers actually want to merge your work.

THE STORY CONTINUES

Arjun is ready to make his first real open-source contribution: fixing a broken example in a popular library's documentation. The commands are ones he already knows – what's new this time is the etiquette of contributing to a project maintained by strangers.



The open-source contribution path, start to finish.

```
terminal

# 1. Fork on GitHub's website first, then:
$ git clone https://github.com/arjunmenon/awesome-library.git
$ cd awesome-library
$ git remote add upstream https://github.com/original-author/awesome-library.git
$ git switch -c fix-readme-typo
# ...edit README.md...
$ git add README.md
$ git commit -m "Fix incorrect installation command in README"
$ git push origin fix-readme-typo
```

Contribution Etiquette

| Do | Why |
|---|---|
| Read the project's <code>CONTRIBUTING.md</code> file, if one exists | Many projects have specific rules about commit style, testing, or branch naming |
| Check open Issues and PRs before starting | Avoids duplicating work someone else is already doing |
| Keep each Pull Request small and focused on one change | Small PRs are far easier and faster for a busy maintainer to review |
| Write a clear PR description explaining what and why | Maintainers often review dozens of PRs from people they've never met |
| Be patient and polite in review comments | Maintainers are frequently volunteers reviewing your work in their free time |

NOTE

Many beginner-friendly projects label small, well-scoped Issues with `good first issue` (Chapter 20) specifically to help newcomers find a safe place to start contributing.

COMMON MISTAKES

- Opening a huge Pull Request that changes dozens of unrelated files at once.
- Never syncing your fork with `upstream`, so your Pull Request ends up based on code that's weeks out of date and conflicts when the maintainer tries to merge it.
- Taking review feedback personally — it's about the code, not you, and responding well to feedback is a skill that makes maintainers want to work with you again.

MINI PRACTICE TASK

Make a real open-source contribution:

1. Find a project with a `good first issue` label.
2. Fork it, clone it, and set up an `upstream` remote.
3. Make a small, focused change on its own branch, and open a Pull Request following the etiquette above.

GitHub Authentication

Proving it's really you, safely

GitHub stopped accepting plain passwords over HTTPS years ago. Here's what replaced them, and the safer alternative many developers prefer instead.

THE STORY CONTINUES

Rahul tries to push using his GitHub username and account password, exactly like logging into a website. It fails. "GitHub doesn't accept your account password for Git operations anymore," Kabir explains. "You need one of two things instead: a Personal Access Token, or an SSH key."

Option 1: HTTPS with a Personal Access Token

THE CONCEPT

A **Personal Access Token (PAT)** is a long, randomly generated string that acts as a substitute password, scoped specifically for Git and API access — generated from your GitHub account settings, copied once, and then used wherever Git prompts for a password.

GENERATING A PERSONAL ACCESS TOKEN

1. On GitHub, go to **Settings** → **Developer settings** → **Personal access tokens**.
2. Generate a new token, choosing an expiry date and the minimum permissions ("scopes") you actually need.
3. Copy the token immediately — GitHub only shows it to you once.
4. The next time Git asks for a password over HTTPS, paste the token instead.

DANGER ZONE

Treat a Personal Access Token exactly like a password: never commit it to a repository, never share it in chat, and never paste it into code. If a token is ever exposed, revoke it immediately from GitHub's settings and generate a new one.

Option 2: SSH Keys

THE CONCEPT

An **SSH key** is a matched pair of files: a **private key** that never leaves your computer, and a **public key** that you upload to GitHub. When you push or pull, your computer proves its identity cryptographically using the private key — without ever typing a password or token again.

ssh-keygen

THE CONCEPT

Generates a new SSH key pair on your computer.

SYNTAX

```
ssh-keygen -t ed25519 -C "your_email@example.com"
```

terminal

```
$ ssh-keygen -t ed25519 -c "rahul.k@example.com"
Generating public/private ed25519 key pair.
Enter file in which to save the key (/home/rahul/.ssh/id_ed25519):
# press Enter to accept the default location, then optionally set a passphrase
Your identification has been saved in /home/rahul/.ssh/id_ed25519
Your public key has been saved in /home/rahul/.ssh/id_ed25519.pub
```

WHEN TO USE THIS

Once per computer you regularly push from.

WHAT GIT DOES INTERNALLY

The `-t ed25519` flag chooses a modern, fast, secure key algorithm. The `-c` flag just attaches a comment (typically your email) to help you recognise the key later — it has no effect on security.

Adding Your SSH Key to GitHub

terminal

```
$ cat ~/.ssh/id_ed25519.pub
ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx rahul.k@example.com
# copy this entire line, then on GitHub:
# Settings -> SSH and GPG keys -> New SSH key -> paste it -> Add SSH key
```

```
git remote set-url origin git@github.com:username/repo.git
```

Switches an already-cloned repository from an HTTPS remote URL to an SSH one, so it starts using your new key.

SYNTAX

```
git remote set-url origin git@github.com:username/repo.git
```

HTTPS vs. SSH: A Quick Recap

| | HTTPS + Token | SSH Key |
|-------------------------------|---|---|
| Setup effort | Very low — generate a token when needed | A bit more upfront setup, once per machine |
| Day-to-day convenience | May need to re-enter a token depending on your credential manager | Effectively invisible once set up — no repeated prompts |
| Good default for | Shared or occasional-use machines (lab computers) | Your own personal laptop |

DANGER ZONE

Whichever method you use, **never commit credentials of any kind** to a repository — tokens, passwords, private keys, or `.env` files full of secrets. Combine `.gitignore` (Chapter 10) with care during `git add` (Chapter 6) as your two lines of defence.

COMMON MISTAKES

- Pasting a Personal Access Token somewhere public, like a commit message or a public chat.
- Sharing a private SSH key file — only the `.pub` file is ever meant to be shared.
- Forgetting a token has expired, and being confused by a sudden authentication failure weeks later.

MINI PRACTICE TASK

Set up secure authentication:

1. Generate a Personal Access Token and use it to push over HTTPS at least once.
2. Generate an SSH key pair, add the public key to your GitHub account, and switch a repository's remote URL to SSH using `git remote set-url`.

GitHub Actions: A Beginner Introduction

Letting GitHub test your code automatically, every single time

Meena wonders if there's a way to make sure nobody accidentally pushes code that breaks the tests. There is — and it runs entirely by itself.

THE STORY CONTINUES

The team has started writing automated tests for the Event Management System. But tests are only useful if someone actually remembers to run them before pushing. "Let's stop relying on remembering," Meena says. "Every time code is pushed, GitHub can automatically run our tests for us."

THE CONCEPT

GitHub Actions is GitHub's built-in automation system. You describe a set of steps in a file, and GitHub runs those steps automatically whenever an event you choose happens — most commonly, every push or every Pull Request. This general idea is called **CI/CD** (Continuous Integration / Continuous Delivery).

Core Vocabulary

| Term | Meaning |
|-----------------|---|
| Workflow | The whole automated process, defined in one YAML file. |
| Event | What triggers the workflow to run — for example, a push, or a Pull Request being opened. |
| Job | A group of steps that run together, on the same virtual machine. |
| Step | One single action within a job — like installing dependencies, or running a test command. |
| Runner | The virtual machine (provided free by GitHub for public repositories) that actually executes the job. |

A Minimal Workflow File

Workflow files live in a specific folder: `.github/workflows/`, and are written in YAML.

```
.github/workflows/tests.yml

name: Run Tests

on: [push, pull_request]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Set up Python
        uses: actions/setup-python@v5
        with:
          python-version: '3.12'
      - name: Install dependencies
        run: pip install -r requirements.txt
      - name: Run tests
        run: pytest
```

WHAT GIT DOES INTERNALLY

`on: [push, pull_request]` is the event. `test:` is the single job, running on an Ubuntu runner. Everything under `steps:` runs in order, top to bottom: checking out the code, setting up Python, installing dependencies, and finally running the test suite.

THE STORY CONTINUES

The next time Rahul opens a Pull Request, GitHub automatically runs this workflow and shows a green checkmark or a red X directly on the PR — letting Meena see at a glance, before ever reading a single line of code, whether Rahul's change broke anything.

WHEN TO USE THIS

GitHub Actions is worth setting up the moment a project has any kind of automated tests, or any repetitive check (like code formatting or linting) that should happen on every change, without anyone having to remember to run it by hand.

COMMON MISTAKES

- Writing a workflow file with incorrect YAML indentation — YAML is extremely sensitive to spacing.
- Forgetting that the file must live under the exact path `.github/workflows/` to be recognised.
- Not checking the Actions tab on GitHub when a check unexpectedly fails, which shows the full log of exactly what went wrong.

MINI PRACTICE TASK

Set up your first workflow:

1. In any repository with at least a basic test, create `.github/workflows/tests.yml` similar to the example above.
2. Push it and open a Pull Request.
3. Watch the Actions tab (and the checkmark on your PR) to see it run automatically.

PART VIII

TROUBLESHOOTING & MASTERY

Common Git Errors

A field guide to the red text every developer eventually sees

Every developer, no matter how experienced, sees these exact messages regularly. Here's how to read them calmly instead of panicking.

THE STORY CONTINUES

Rahul calls Kabir over, convinced he has "broken Git." Kabir glances at the screen and laughs: "This isn't broken. Git is actually telling you exactly what's "wrong, in plain English. Let's just read it together." This chapter collects the errors every one of this book's characters has hit at some point — so that when you see them, you'll already know exactly what to do.

fatal: not a git repository (or any of the parent directories): .git

| | |
|--------------------------|---|
| What it means | Git commands only work inside a folder that has been initialized as a repository. |
| Why it happened | You ran a Git command in a folder that has never had <code>git init</code> run in it, or that you cloned into a different location than you think. |
| How to fix it | Run <code>pwd</code> (or <code>cd</code> alone on Windows) to check your current folder, then either <code>cd</code> into the correct project folder, or run <code>git init</code> if this really is a brand new project. |
| How to prevent it | Get in the habit of checking which folder your terminal is in before running Git commands, especially right after opening a new terminal window. |

nothing to commit, working tree clean

| | |
|--------------------------|---|
| What it means | This is not an error at all — it's Git confirming that every file matches your last commit exactly, with nothing new to save. |
| Why it happened | You ran <code>git commit</code> without having staged (or made) any changes first. |
| How to fix it | If you expected changes to exist, first run <code>git status</code> to check whether you actually edited the file you think you did, and whether it needs <code>git add</code> first. |
| How to prevent it | Run <code>git status</code> before every commit as a habit, so this is never a surprise. |

! [rejected] main -> main (non-fast-forward)

| | |
|--------------------------|---|
| What it means | The remote branch has commits that your local branch doesn't have, so Git refuses to push and silently overwrite them. |
| Why it happened | A teammate pushed to the same branch after your last <code>pull</code> , and your local branch has fallen behind. |
| How to fix it | Run <code>git pull</code> to bring in the missing commits (resolving any conflict if one appears, Chapter 17), then push again. |
| How to prevent it | Pull before you start working, and pull again right before you push, especially on shared branches like <code>main</code> . |

fatal: remote origin already exists.

| | |
|--------------------------|---|
| What it means | A remote named <code>origin</code> is already registered for this repository. |
| Why it happened | You ran <code>git remote add origin <url></code> a second time, perhaps after following setup instructions twice. |
| How to fix it | Either remove the existing one first with <code>git remote remove origin</code> and re-add it, or simply update it in place with <code>git remote set-url origin <url></code> . |
| How to prevent it | Run <code>git remote -v</code> first to check whether a remote is already configured before adding one. |

Your branch is ahead of 'origin/main' by 2 commits.

| | |
|--------------------------|--|
| What it means | You have local commits that exist only on your machine — they haven't been pushed yet. |
| Why it happened | You committed locally but haven't run <code>git push</code> since. |
| How to fix it | Run <code>git push</code> to upload those commits, if that's what you intend to do. |
| How to prevent it | This isn't really an error to prevent — it's <code>git status</code> keeping you informed. Just remember to push when you're ready to share your work. |

Your branch is behind 'origin/main' by 3 commits, and can be fast-forwarded.

| | |
|--------------------------|---|
| What it means | The remote has commits your local branch doesn't have yet, and they can be safely, automatically merged in. |
| Why it happened | A teammate pushed new commits since you last pulled. |
| How to fix it | Run <code>git pull</code> to catch up. |
| How to prevent it | Make pulling the first thing you do at the start of every work session on a shared branch. |

CONFLICT (content): Merge conflict in <file>

| | |
|--------------------------|---|
| What it means | Two branches changed the same lines of the same file in different ways, and Git needs you to decide which version (or combination) is correct. |
| Why it happened | You merged, pulled, or rebased and the incoming changes overlap with your own uncommitted or committed local changes. |
| How to fix it | Open the file, resolve the conflict markers by hand as shown in Chapter 17, then <code>git add</code> the file and <code>git commit</code> to finish. |
| How to prevent it | Pull frequently so your branch never drifts too far from <code>main</code> before merging, and communicate with teammates editing the same files. |

Permission denied (publickey).

| | |
|--------------------------|---|
| What it means | GitHub rejected an SSH connection because it doesn't recognise your computer's SSH key. |
| Why it happened | You're using an SSH remote URL, but either no SSH key has been generated on this machine, or it hasn't been added to your GitHub account yet. |
| How to fix it | Follow the SSH key setup steps in Chapter 28: generate a key with <code>ssh-keygen</code> and add the public key to your GitHub account's SSH settings. |
| How to prevent it | Set up your SSH key once per new computer, before you need it urgently. |

remote: Support for password authentication was removed. ... Authentication failed

| | |
|--------------------------|--|
| What it means | GitHub no longer accepts your GitHub account password for Git operations over HTTPS. |
| Why it happened | You typed your regular GitHub password when Git prompted for one, instead of a Personal Access Token. |
| How to fix it | Generate a Personal Access Token (Chapter 28) and use that in place of a password. |
| How to prevent it | Consider switching to SSH authentication instead, so you're never prompted for a password or token at all. |

TIP

A general-purpose debugging habit that solves a surprising number of these errors on its own: run `git status` first, read every line of its output carefully, and only then decide what to do next.

MINI PRACTICE TASK

Build your troubleshooting muscle:

1. Intentionally trigger at least two of the errors above in a throwaway test repository (for example, run a Git command outside any repository, or push after a teammate/second clone has pushed first).
2. Practice reading the error message fully before searching for a solution online.
3. Fix each one using the steps in this chapter.

Git Best Practices

The habits that separate a confident developer from a dangerous one

Kabir's final piece of advice before letting the team loose on their own: commands are easy to look up, but good habits are what actually prevent disasters.

THE STORY CONTINUES

With finals approaching and the project nearly done, Kabir gathers the team one last time. "You all know the commands now," he says. "What separates a "confident developer from a dangerous one isn't knowing more commands – it's a handful of habits. Here's everything I actually want you to remember."

| Practice | Why it matters |
|--|---|
| Commit frequently | Small, frequent commits create fine-grained checkpoints you can return to – the opposite of Arjun's zip-file folder from Chapter 1. |
| Make small, atomic commits | Each commit should represent one logical change, making history easy to read, review, and undo selectively (Chapter 7). |
| Write meaningful commit messages | A good message tells future-you (and everyone else) what changed and why, without having to read the code. |
| Pull before starting work | Starting from an up-to-date <code>main</code> avoids painful, avoidable conflicts and rejected pushes later. |
| Use feature branches | Keeps unfinished, experimental work completely isolated from the stable, working version of the project. |
| Never commit secrets | A committed secret lives in history forever, even after you delete the file in a later commit (Chapter 10). |
| Use .gitignore from day one | Prevents dependency folders, build artifacts, and machine-specific files from ever being tracked in the first place. |
| Review changes before committing | A quick <code>git status</code> and <code>git diff</code> before every commit catches mistakes before they enter history. |
| Don't blindly copy commands from the internet | Understand what a command does – especially anything involving <code>--force</code> , <code>--hard</code> , or <code>rm</code> – before running it on a real project. |
| Don't force-push carelessly | Force-pushing can silently overwrite and destroy a teammate's commits on a shared branch. |
| Don't rewrite shared history | Rebasing or resetting commits others have already pulled creates painful, confusing divergence (Chapters 9 and 24). |
| Keep the README updated | An outdated README actively misleads the very people it's supposed to help (Chapter 21). |

THE CONCEPT

If you remember nothing else from this entire book, remember this: **Git rewards small, frequent, well-described actions, and punishes big, rare, vague ones.** Small commits, small Pull Requests, frequent pulls, clear messages – every single best practice in this chapter is really just that one idea, applied to a different command.

MINI PRACTICE TASK

Turn this chapter into a personal checklist:

1. Pick the three habits above you're currently weakest at.
2. For your next project, deliberately practice just those three until they feel automatic.
3. Revisit this list again in a month and see how many have become second nature.

PART IX

REFERENCE

Git Command Cheat Sheet

Every command in this book, one line each

For the week before a lab exam, or the moment your mind goes blank mid-project – this is the page to bookmark.

Setup

| Command | One-line explanation |
|---|--|
| <code>git --version</code> | Check that Git is installed and see its version. |
| <code>git config --global user.name/user.email</code> | Tell Git your identity for every commit. |

Create

| Command | One-line explanation |
|------------------------------------|---|
| <code>git init</code> | Turn the current folder into a brand-new, empty repository. |
| <code>git clone <url></code> | Download a complete copy of an existing repository. |

Status

| Command | One-line explanation |
|-------------------------|--|
| <code>git status</code> | Show exactly what's changed, staged, or untracked right now. |

Staging

| Command | One-line explanation |
|-----------------------------------|--|
| <code>git add <file></code> | Mark a specific file's changes as ready for the next commit. |
| <code>git add .</code> | Stage every changed and new file in the current folder. |

Commit

| Command | One-line explanation |
|----------------------------------|--|
| <code>git commit -m "msg"</code> | Permanently save staged changes as a new snapshot. |
| <code>git commit --amend</code> | Modify your most recent (not-yet-pushed) commit. |

History

| Command | One-line explanation |
|--|--|
| <code>git log</code> / <code>--oneline</code> / <code>--graph</code> | View the project's commit history in varying levels of detail. |
| <code>git show <commit></code> | See full details and changes for one specific commit. |
| <code>git diff</code> / <code>--staged</code> | Compare working directory, staging area, and last commit. |

Branches

| Command | One-line explanation |
|------------------------------|---|
| <code>git branch</code> | List, create, or delete branches. |
| <code>git switch / -c</code> | Change to a branch, or create and switch in one step. |

Merging

| Command | One-line explanation |
|---------------------------------------|---|
| <code>git merge <branch></code> | Combine another branch's history into your current one. |

Remote

| Command | One-line explanation |
|--|---|
| <code>git remote -v</code> | List configured remotes and their URLs. |
| <code>git remote add <name> <url></code> | Register a new remote connection. |

Sync

| Command | One-line explanation |
|------------------------|--|
| <code>git fetch</code> | Download remote changes without merging them into your branch. |
| <code>git pull</code> | Fetch and immediately merge remote changes. |
| <code>git push</code> | Upload your local commits to a remote repository. |

Undo

| Command | One-line explanation |
|--|--|
| <code>git restore <file></code> | Discard uncommitted changes in the working directory. |
| <code>git revert <commit></code> | Undo a commit safely with a new, opposite commit. |
| <code>git reset --soft/mixed/hard</code> | Rewind history, keeping progressively less of the undone work. |

Temporary

| Command | One-line explanation |
|------------------------------|---|
| <code>git stash / pop</code> | Set aside uncommitted work temporarily, then bring it back later. |

Tags

| Command | One-line explanation |
|---|---|
| <code>git tag -a v1.0.0 -m "msg"</code> | Permanently mark the current commit as a named release point. |

TIP

This cheat sheet intentionally omits every explanation, warning, and example from earlier chapters. If any line here doesn't immediately make sense on its own, that chapter is exactly where to go back and refresh your understanding — this page is a reminder, not a replacement, for actually knowing what each command does.

Command-by-Command Reference

One detailed table, every command, side by side

Where the cheat sheet is fast, this chapter is thorough – the full picture for every command taught in this book, in one place.

Risk level reflects how easily a command can cause accidental, hard-to-undo damage if used carelessly: **LOW** commands are safe to run freely, **MEDIUM** commands deserve a moment's thought, and **HIGH** commands can destroy uncommitted or even committed work if misused.

| Command | Purpose | Syntax | Example | When to use | Risk | Common mistake |
|--------------------------|--------------------------------|--|---|---|--------|---|
| <code>git init</code> | Start a new repository | <code>git init</code> | <code>git init</code> | New local project | LOW | Running inside an existing repo |
| <code>git clone</code> | Download an existing repo | <code>git clone <url></code> | <code>git clone https://github.com/x/y.git</code> | Joining an existing project | LOW | Cloning into a non-empty folder |
| <code>git status</code> | Show current file states | <code>git status</code> | <code>git status</code> | Constantly, before/after other commands | LOW | Skipping it and guessing instead |
| <code>git add</code> | Stage changes | <code>git add <file> / .</code> | <code>git add app.py</code> | Before every commit | LOW | Blind <code>git add .</code> without checking status |
| <code>git commit</code> | Save a permanent snapshot | <code>git commit -m "msg"</code> | <code>git commit -m "Fix bug"</code> | After completing one logical change | LOW | Vague messages; giant mixed commits |
| <code>git log</code> | View commit history | <code>git log [--oneline --graph]</code> | <code>git log --oneline</code> | Investigating past changes | LOW | Not knowing <code>q</code> exits the pager |
| <code>git show</code> | Inspect one commit fully | <code>git show <commit></code> | <code>git show 8b3e1af</code> | After finding a commit via log | LOW | Confusing it with <code>git log</code> |
| <code>git diff</code> | Compare versions | <code>git diff [--staged]</code> | <code>git diff --staged</code> | Reviewing changes before staging/committing | LOW | Not knowing which of the two modes you're seeing |
| <code>git restore</code> | Discard/unstage changes | <code>git restore [--staged] <file></code> | <code>git restore app.py</code> | Throwing away unwanted edits | MEDIUM | Discarding changes you actually wanted to keep |
| <code>git reset</code> | Rewind branch history | <code>git reset --soft/mixed/hard HEAD~1</code> | <code>git reset --hard HEAD~1</code> | Undoing local, unpushed commits | HIGH | Using <code>--hard</code> without understanding data loss |
| <code>git revert</code> | Safely undo a shared commit | <code>git revert <commit></code> | <code>git revert 8b3e1af</code> | Undoing a pushed/shared commit | LOW | Using <code>reset</code> instead on shared history |
| <code>git branch</code> | List/create/delete branches | <code>git branch [-d -D] [name]</code> | <code>git branch -d old-feature</code> | Managing parallel lines of work | LOW | Using <code>-D</code> out of habit instead of <code>-d</code> |
| <code>git switch</code> | Change branches | <code>git switch [-c] <branch></code> | <code>git switch -c feature-x</code> | Moving between branches | LOW | Switching with uncommitted conflicting changes |
| <code>git merge</code> | Combine branch histories | <code>git merge <branch></code> | <code>git merge feature-login</code> | Bringing finished work into main | MEDIUM | Merging into the wrong current branch |
| <code>git rebase</code> | Replay commits onto a new base | <code>git rebase <branch></code> | <code>git rebase main</code> | Cleaning up a personal branch before a PR | HIGH | Rebasing already-shared/pushed commits |
| <code>git stash</code> | Temporarily shelve changes | <code>git stash [pop list apply drop]</code> | <code>git stash pop</code> | Switching context without committing | LOW | Forgetting stashed work exists |
| <code>git remote</code> | Manage remote connections | <code>git remote add/rm/set-url</code> | <code>git remote add origin <url></code> | Connecting to GitHub or additional remotes | LOW | Adding a remote that already exists |
| <code>git fetch</code> | Download remote changes only | <code>git fetch [origin]</code> | <code>git fetch origin</code> | Reviewing remote changes before integrating | LOW | Expecting files to update automatically |
| <code>git pull</code> | Fetch + merge in one step | <code>git pull [origin main]</code> | <code>git pull</code> | Starting a work session on a shared branch | MEDIUM | Pulling with unresolved local conflicts |
| <code>git push</code> | Upload local commits | <code>git push [-u origin]</code> | <code>git push -u origin main</code> | Sharing/backing up | MEDIUM | Force-pushing over a |

| | | main] | | committed work | | teammate's commits |
|-------------------------|------------------------------------|--|---|-----------------------------|-----|--|
| <code>git tag</code> | Mark a specific commit permanently | <code>git tag -a v1.0.0 -m "msg"</code> | <code>git tag -a v1.0.0 -m "Release"</code> | Marking an official release | LOW | Using lightweight tags for real releases |
| <code>git config</code> | Read/change Git settings | <code>git config --global <key> <value></code> | <code>git config --global user.name "Me"</code> | One-time machine setup | LOW | Forgetting <code>--global</code> and repeating setup per project |

PART X

PRACTICE & ASSESSMENT

Hands-On Labs

Ten progressively harder exercises, from your first commit to a real contribution

Reading about Git only gets you so far. These ten labs are where it actually becomes muscle memory.

LAB 1: Create Your First Git Repository

OBJECTIVE

Objective: Initialize a repository from scratch and make your first commit.

PREREQUISITES

Prerequisites: Git installed and configured (Chapter 2).

STEP-BY-STEP

1. Create a new folder and move into it.
2. Run `git init`.
3. Create a `README.md` file with a one-line project description.
4. Run `git status`, then `git add README.md`.
5. Run `git commit -m "Initial commit"`.
6. Run `git log --oneline` to confirm your commit exists.

WHEN TO USE THIS

Expected result: A local repository with exactly one commit, and a clean working tree.

CHALLENGE

Challenge (optional, harder): Add a `.gitignore` file and a second file, then make a second, separate commit.

COMMON MISTAKES

- Forgetting `git add` before `git commit`.
- Running `git init` in the wrong folder.

LAB 2: Track and Commit a Simple Project

OBJECTIVE

Objective: Practice the everyday edit → status → add → commit rhythm across multiple files.

PREREQUISITES

Prerequisites: Completion of Lab 1.

STEP-BY-STEP

1. In your Lab 1 repository, create two source files (for example, `main.py` and `utils.py`).
2. Write a small amount of working code in each.
3. Stage and commit them as one commit.
4. Edit only `main.py` again, and commit just that change separately.
5. Run `git log --oneline` and confirm you now have three commits total.

WHEN TO USE THIS

Expected result: A short, readable commit history where each commit represents one clear, separate change.

CHALLENGE

Challenge (optional, harder): Use `git add` with specific filenames throughout this lab instead of ever using `git add .`

COMMON MISTAKES

- Combining unrelated changes into a single commit.
- Vague commit messages like "update" instead of describing what changed.

LAB 3: Connect the Project to GitHub

OBJECTIVE

Objective: Publish a local repository to GitHub for the first time.

PREREQUISITES

Prerequisites: A GitHub account, and completion of Lab 2.

STEP-BY-STEP

1. Create a new, empty repository on GitHub (do not initialize it with a README).
2. In your local project, run `git remote add origin <url>` using that repository's URL.
3. Run `git branch -M main`.
4. Run `git push -u origin main`.
5. Refresh the GitHub page and confirm your files and commit history appear.

WHEN TO USE THIS

Expected result: Your local repository's full history visible on GitHub.com.

CHALLENGE

Challenge (optional, harder): Edit the README directly on GitHub's website, then run `git pull` locally to bring that change back down.

COMMON MISTAKES

- Checking "Initialize with a README" on GitHub when you already have local commits.
- Copying the wrong repository URL.

LAB 4: Create and Use a Feature Branch

OBJECTIVE

Objective: Practice isolating new work on its own branch.

PREREQUISITES

Prerequisites: Completion of Lab 3.

STEP-BY-STEP

1. Run `git switch -c add-logging`.
2. Add a small logging feature to one of your files.
3. Commit it on this branch.
4. Push the branch: `git push origin add-logging`.
5. Switch back to `main` and confirm the change is not present there.

WHEN TO USE THIS

Expected result: A pushed feature branch, separate from main, containing your new commit.

CHALLENGE

Challenge (optional, harder): Create a second branch from `main` at the same time, make an unrelated change, and keep both branches pushed simultaneously without mixing their commits.

COMMON MISTAKES

- Committing the new feature directly to `main` instead of a branch.
- Forgetting which branch is currently checked out.

LAB 5: Create and Merge a Pull Request

OBJECTIVE

Objective: Go through a complete, real Pull Request from open to merge.

PREREQUISITES

Prerequisites: Completion of Lab 4, with a pushed feature branch.

STEP-BY-STEP

1. On GitHub, open a Pull Request comparing your feature branch to `main`.
2. Write a clear title and description.
3. If possible, ask a classmate to review it and leave at least one comment.
4. Make any requested change, commit, and push again.
5. Once approved, click **Merge pull request** on GitHub.
6. Locally, run `git switch main` and `git pull`.

WHEN TO USE THIS

Expected result: Your feature branch's commits are now part of main's history, visible with `git log --graph --oneline`.

CHALLENGE

Challenge (optional, harder): Try all three merge strategies (merge commit, squash, rebase) on three different small PRs and compare the resulting history.

COMMON MISTAKES

- Merging your own PR without any review.
- Forgetting to pull main locally after merging on GitHub.

LAB 6: Intentionally Create and Resolve a Merge Conflict

OBJECTIVE

Objective: Build confidence resolving conflicts by causing one on purpose, in a safe environment.

PREREQUISITES

Prerequisites: Comfort with branching and merging (Labs 4-5).

STEP-BY-STEP

1. From `main`, create two branches: `branch-a` and `branch-b`.
2. On each branch, edit the exact same line of the exact same file, in different ways.
3. Commit on both branches.
4. Merge `branch-a` into `main` (should succeed cleanly).
5. Merge `branch-b` into `main` and confirm you get a CONFLICT message.
6. Open the conflicted file, resolve it by hand, remove the markers, then `git add` and `git commit`.

WHEN TO USE THIS

Expected result: A successfully completed merge commit, with the conflict fully and correctly resolved.

CHALLENGE

Challenge (optional, harder): Trigger a conflict, then practice `git merge --abort` to cancel it cleanly before trying again for real.

COMMON MISTAKES

- Leaving conflict markers in the committed file by accident.
- Deleting the whole file instead of resolving the conflict inside it.

LAB 7: Use git stash

OBJECTIVE

Objective: Practice safely shelving unfinished work to switch context.

PREREQUISITES

Prerequisites: A repository with at least one branch besides main.

STEP-BY-STEP

1. Make an uncommitted, incomplete edit to a tracked file.
2. Run `git stash push -m "WIP test"`.
3. Confirm `git status` is clean, and switch to a different branch and back.
4. Run `git stash list` to see your saved stash.
5. Run `git stash pop` to bring your changes back.

WHEN TO USE THIS

Expected result: Your incomplete edit returns exactly as you left it, after safely switching branches in between.

CHALLENGE

Challenge (optional, harder): Create two separate stashes with different messages, then use `git stash apply stash@{1}` to apply the older one specifically.

COMMON MISTAKES

- Forgetting a stash exists.
- Popping a stash onto a branch where it immediately conflicts.

LAB 8: Undo a Mistake Safely

OBJECTIVE

Objective: Practice choosing the correct undo tool for different situations.

PREREQUISITES

Prerequisites: Comfort with commits (Lab 2) and Chapter 9.

STEP-BY-STEP

1. Make an uncommitted edit you don't want, and discard it with `git restore`.
2. Stage a file, then unstage it with `git restore --staged` without losing the edit.
3. Make a commit, then undo it while keeping the changes staged, using `git reset --soft HEAD~1`.
4. Make another commit, push it, then safely undo it with `git revert` instead of `reset`.

WHEN TO USE THIS

Expected result: Four different "mistakes" corrected using four different, situation-appropriate commands.

CHALLENGE

Challenge (optional, harder): Deliberately use `git reset --hard` on a throwaway commit and confirm for yourself that the change is genuinely, permanently gone.

COMMON MISTAKES

- Reaching for `git reset --hard` as a default undo button.
- Using `reset` on a commit that's already been pushed and pulled by someone else.

LAB 9: Collaborate With Another Student

OBJECTIVE

Objective: Experience a real two-person Git workflow, conflicts and all.

PREREQUISITES

Prerequisites: A willing classmate, and a shared GitHub repository you both have access to.

STEP-BY-STEP

1. One of you creates a repository and adds the other as a collaborator (or you both fork/clone the same public repository).
2. Each of you creates your own feature branch and works independently for 10-15 minutes.
3. Push both branches and open Pull Requests.
4. Review each other's PR with at least one real comment each.
5. Merge both, pull the results locally, and resolve any conflict that appears when the second PR merges.

WHEN TO USE THIS

Expected result: Two independently developed features, both successfully merged into a shared main branch.

CHALLENGE

Challenge (optional, harder): Deliberately have both of you edit the same function at the same time to guarantee a conflict, then resolve it together over a call or in person.

COMMON MISTAKES

- Not communicating about which files each person is working on.
- One person force-pushing over the other's work.

LAB 10: A Mini Open-Source Contribution

OBJECTIVE

Objective: Complete the entire fork-to-merge journey on a real public project.

PREREQUISITES

Prerequisites: Comfort with Chapters 18, 19, and 27.

STEP-BY-STEP

1. Find a small, beginner-friendly open-source repository (look for a "good first issue" label).
2. Fork it and clone your fork.
3. Add the original repository as an **upstream** remote.
4. Create a branch and make one small, focused improvement.
5. Push to your fork and open a Pull Request to the original repository.
6. Respond politely and promptly to any review feedback.

WHEN TO USE THIS

Expected result: A submitted, real-world Pull Request to a project you don't personally own.

CHALLENGE

Challenge (optional, harder): Find and complete a second contribution to a different project, applying everything you learned from the first one's review feedback.

COMMON MISTAKES

- Opening an overly large, unfocused Pull Request.
- Never syncing your fork with **upstream**, leading to an out-of-date PR.

Mini Project: College Club Management System

One assignment that exercises everything this book has taught

This is the final assignment Kabir gives the team before they graduate from ‘Git beginners’ to ‘developers who happen to know Git.’

THE STORY CONTINUES

"Forget separate exercises," Kabir says. "Your last assignment is one "project that needs every single thing you've learned. Build it properly, using real Git and GitHub workflow from the very first commit."

THE CONCEPT

Project: A **College Club Management System** — a simple application where students can view college clubs, join them, and club coordinators can post announcements and events. The exact features and tech stack are your choice; what's being assessed here is your **Git and GitHub process**, not the app itself.

Requirements Checklist

| # | Requirement |
|----|---|
| 1 | Create a GitHub repository for the project. |
| 2 | Add a complete, professional <code>README.md</code> (Chapter 21's structure). |
| 3 | Add a <code>.gitignore</code> appropriate for your chosen stack (Chapter 10). |
| 4 | Set up <code>main</code> as the default branch. |
| 5 | Create at least three feature branches for three different pieces of functionality. |
| 6 | Make meaningful, atomic commits with clear messages throughout (Chapter 7). |
| 7 | Push every feature branch to GitHub. |
| 8 | Open a Pull Request for each feature branch. |
| 9 | Review at least one teammate's Pull Request with real comments (or self-review with detailed notes, if working solo). |
| 10 | Intentionally cause and properly resolve at least one real merge conflict. |
| 11 | Merge all completed features into <code>main</code> . |
| 12 | Create an annotated tag <code>v1.0.0</code> once the project is stable. |
| 13 | Publish a GitHub Release based on that tag. |

Evaluation Rubric

| Criterion | What's being checked | Points |
|----------------------|---|--------|
| Repository setup | README quality, .gitignore correctness, sensible project structure | 15 |
| Commit quality | Atomic commits, clear imperative-mood messages, logical history | 20 |
| Branching discipline | Feature branches used correctly, no direct commits to main for features | 15 |
| Pull Requests | Clear descriptions, at least one real review exchange visible | 20 |

| | | |
|-----------------------------|--|----|
| Conflict resolution | A genuine conflict was created and correctly, cleanly resolved | 15 |
| Release | A correctly annotated tag and a matching GitHub Release with notes | 10 |
| Overall history readability | Running <code>git log --graph --oneline --all</code> tells a clear, sensible story | 5 |

NOTE

Total: 100 points. As a benchmark, a project that mixes several unrelated changes into single commits, never uses Pull Requests, or has no branches beyond `main` at all is missing most of what this assignment is actually testing — regardless of how polished the final application looks.

MINI PRACTICE TASK

Before submitting, verify these for yourself:

1. Run `git log --graph --oneline --all` and read your own history as if you were a stranger — does it make sense?
2. Confirm your README's Installation section actually works by following it exactly, from scratch.
3. Confirm no secrets, passwords, or API keys appear anywhere in your commit history.
4. Confirm your `v1.0.0` release is visible under your repository's Releases tab.

Final Interview / Viva Questions

Forty questions that separate 'used Git once' from 'understands Git'

Whether it's a college viva, a lab exam, or your first technical interview, these are the questions that come up again and again.

Answers are provided immediately after each question so you can use this chapter both as practice questions and as a fast final review.

Q1. What is Git?

A free, open-source Distributed Version Control System (DVCS) that tracks changes to files over time, installed and run entirely on your own computer.

Q2. What is GitHub?

A web-based platform that hosts Git repositories online and adds collaboration features on top of Git, such as Pull Requests, Issues, and code review.

Q3. What is the difference between Git and GitHub?

Git is a tool/software that runs locally and works offline; GitHub is a website built on top of Git that requires the internet and adds collaboration features.

Q4. What is a repository?

A project folder that Git is tracking, containing both the current files and their entire history of changes.

Q5. What is a commit?

A permanent, saved snapshot of the project at one point in time, with a message, author, timestamp, and unique hash.

Q6. What is staging?

An intermediate step where you mark exactly which changes should be included in the next commit, using `git add`.

Q7. Why use `git add` before `git commit`?

It gives you fine-grained control over exactly which changes enter each commit, instead of being forced to commit everything that changed at once.

Q8. What is the difference between `git pull` and `git fetch`?

`git fetch` downloads remote changes without touching your working directory; `git pull` does the same download and then immediately merges those changes into your current branch.

Q9. What is the difference between `git merge` and `git rebase`?

Both integrate one branch's changes into another, but merge preserves history exactly as it happened (via a merge commit), while rebase rewrites history into a straight line by replaying commits onto a new base.

Q10. What is the difference between `git revert` and `git reset`?

`git revert` undoes a commit by adding a new, opposite commit, leaving history intact and safe for shared branches; `git reset` moves the branch pointer backward, rewriting history, and is only safe on commits nobody else has pulled.

Q11. What is HEAD?

A pointer to your current position in the repository's history — usually the latest commit on whichever branch you have checked out.

Q12. What is origin?

The conventional nickname for the main remote a repository was cloned from, or connected to first.

Q13. What is a branch?

An independent, parallel line of development that lets you make commits without affecting other branches until you choose to merge.

Q14. What is a Pull Request?

A GitHub feature for proposing that changes from one branch be merged into another, with space for review and discussion before the merge happens.

Q15. What is a merge conflict?

A situation where two branches have changed the same lines of the same file in incompatible ways, requiring a human to manually decide the final result.

Q16. Why use a .gitignore file?

To tell Git which files and folders (like dependencies, build output, or secrets) should never be tracked, even though they exist on disk.

Q17. Why should API keys never be committed?

Because once committed, they exist permanently in the project's history — accessible to anyone who can view the repository — even if the file is deleted in a later commit.

Q18. What is a fork?

Your own complete, independent copy of someone else's repository, created on GitHub, that you have full write access to even without permission on the original.

Q19. What is the difference between fork and clone?

A fork is a website action on GitHub that creates a new repository under your account; a clone is a terminal action that downloads any repository (including a fork) to your own computer.

Q20. What is GitHub Actions?

GitHub's built-in automation system (CI/CD) that runs a defined set of steps automatically in response to events like a push or a Pull Request.

Q21. What does git init do?

Turns the current folder into a new, empty Git repository by creating a hidden `.git` folder.

Q22. What does git clone do differently from git init?

`git clone` downloads a complete copy of an existing repository's history from elsewhere; `git init` creates a brand-new, empty one with no history.

Q23. What is the staging area also called?

The index.

Q24. What are the four states a file can be in?

Untracked, Modified, Staged, and Committed.

Q25. What does "working tree clean" mean?

That every file currently on disk exactly matches the last commit, with nothing new to stage or commit.

Q26. What is an atomic commit?

A commit that contains exactly one logical, self-contained change.

Q27. What does git commit --amend do?

Replaces your most recent, not-yet-pushed commit with a new one, letting you fix its message or add forgotten changes.

Q28. What is the difference between git log and git log --oneline?

`git log` shows full details (hash, author, date, message) per commit; `--oneline` condenses each commit to a single line.

Q29. What does git diff show by default?

The differences between your working directory and the staging area — your unstaged changes.

Q30. What is the difference between git diff and git diff --staged?

`git diff` shows unstaged changes; `git diff --staged` shows changes that are staged and about to be committed.

Q31. What is the difference between git restore and git reset?

`git restore` operates on the working directory and staging area to discard or unstage uncommitted changes; `git reset` moves which commit your branch pointer refers to.

Q32. What does git reset --hard do, and why is it risky?

It moves the branch pointer backward and permanently deletes the corresponding changes from the working directory too — risky because that deletion cannot be undone.

Q33. What is the modern replacement for git checkout when switching branches?

`git switch`.

Q34. What is the difference between git branch -d and git branch -D?

`-d` only deletes a branch if it has already been merged (a safe delete); `-D` forces deletion even if it hasn't been merged, discarding any unmerged commits.

Q35. What is a fast-forward merge?

A merge where the current branch hasn't moved since the other branch split off, so Git simply moves the branch pointer forward with no new commit needed.

Q36. What is a three-way merge?

A merge where both branches have diverged, so Git creates a new merge commit with two parents, combining both histories using their common ancestor as reference.

Q37. What are the conflict markers <<<<<<, =====, and >>>>>> for?

They mark the boundaries between the two conflicting versions of the code during a merge conflict, so you can see and manually resolve both sides.

Q38. What does git stash do?

Temporarily saves uncommitted changes aside, restoring a clean working directory, so they can be brought back later with `git stash pop` or `apply`.

Q39. Why is rewriting shared history with rebase or reset dangerous?

Because teammates who already pulled the old commits will have a history that has permanently diverged from the new one, causing confusing conflicts and duplicated commits when reconciling.

Q40. What is the purpose of an upstream remote?

In a forked repository, `upstream` conventionally refers to the original project, letting you fetch and merge its latest changes into your own fork, which is tracked separately as `origin`.

Q41. What is semantic versioning?

A version numbering convention, MAJOR.MINOR.PATCH, where MAJOR increments for breaking changes, MINOR for new backward-compatible features, and PATCH for backward-compatible bug fixes.